

PORTFOLIO COURSEWORK 2

Question 1:

**Option 2: Real World Data Analysis & Business Problem
Solving**

E-Commerce Shipping Delay Prediction

Question 2:

***Compare and Contrast the Following Probability
Distributions***

Git repository: <https://github.com/AnuradhaDhananjane/Portfolio-CW2>

Name: Anuradha Dhananjane

Student ID: COMScDS252P-005

Course: Principles of Data Science

Date: 05/04/2026

Contents

Table of figures.....	5
Table of tables	6
1. Question 01-Option 02	7
1. Data Collection	7
1.1 Data set overview	7
1.2 Data Collection Methodology.....	7
1.3 Column Descriptions	8
1.4 Cloud Integration (AWS).....	9
2. Exploratory Data Analysis (EDA).....	10
2.1 Data Cleaning & Preprocessing	10
2.2 Missing Values & Outliers.....	10
2.3 Descriptive Statistics.....	12
2.4 Key Visualizations	13
2.5 Correlation Heatmap.....	20
2.6 Feature Engineering.....	21
2.7 Business Insights from EDA.....	22
3. Clustering Analysis	22
3.1 Methodology.....	22
3.2 Optimal K — Elbow Method.....	22
3.3 Cluster Results	24
3.4 Clustering Insights	27
4. Problem Solving & Predictive Model	28
4.1 Business Problem Definition	28
4.2 Model Development	28
4.3 Model Performance Evaluation	30
4.4 Feature Importance.....	33
5. Business Impact Analysis	34

5.1	Current Business Problem	34
5.2	Impact of Model Implementation	34
5.3	Operational Improvement Recommendations	35
5.4	Scalability Considerations	35
5.5	Web Application & Interactive Dashboard	36
6.	Cloud Architecture Overview (AWS)	38
6.1	Architecture Design	38
6.2	Data Pipeline Architecture	38
6.3	Cloud Setup Note.....	38
7.	Collaboration & Documentation	40
7.1	Git Repository Structure	40
7.2	Git Commit Strategy	40
7.3	Assumptions & Decisions	41
7.4	Challenges & Solutions.....	42
8.	Conclusion	43
	References	43
2.	Question 02	44
1.	Introduction	44
2.	Comparison Table	44
3.	Individual Distributions — Definition and Characteristics	46
a.	Binomial Distribution	46
b.	Poisson Distribution	46
c.	Geometric Distribution	47
d.	Hypergeometric Distribution.....	48
e.	Negative Binomial Distribution	48
f.	Exponential Distribution	49
g.	Normal Distribution.....	50
h.	Chi-Square Distribution.....	50
4.	Similarities Across All Distributions	52

5. Compare and Contrast	53
5.1 First Major Contrast — Discrete vs Continuous.....	53
5.2 Comparing the Five Discrete Distributions	53
5.3 Comparing the Three Continuous Distributions	54
5.4 Cross-Family Relationships (Discrete $\leftrightarrow$ Continuous)	55
6. Summary of Key Similarities and Differences	55
7. Conclusion	56
References	57

Table of figures

Figure 1 - Column names and data types	8
Figure 2 - AWS Console home page	9
Figure 3 - Missing values	10
Figure 4 - Outlier Detection	11
Figure 5 - Outlier detection - Boxplots	11
Figure 6 - Delivery status - Coding	12
Figure 7 - Delivery status distribution	13
Figure 8 - Shipment mode vs late delivery code	14
Figure 9 - Shipment mode vs late delivery Graph.....	14
Figure 10 - Warehouse block vs late delivery code	15
Figure 11 - Warehouse block vs late delivery Block	15
Figure 12 - Product importance vs late delivery code	16
Figure 13 - Product importance vs late delivery code chart	16
Figure 14 - Weight distribution code.....	17
Figure 15 - Weight distribution chart	17
Figure 16 - Discount vs late delivery code.....	18
Figure 17 - Discount vs late delivery chart	18
Figure 18 - Customer care calls vs late delivery code	19
Figure 19 - Customer care calls vs late delivery status	19
Figure 20 - Heatmap code.....	20
Figure 21 - Correlation Heatmap	21
Figure 22 - Optimal K — Elbow Method	23
Figure 23 - Optimal K — Elbow Method chart.....	23
Figure 24- Weight vs product cost scatterplot coding.....	24
Figure 25 - Weight vs product cost scatterplot	25
Figure 26 - cluster profile bar chart coding.....	25
Figure 27- cluster profile bar chart.....	26
Figure 28 - Late delivery rate per cluster code.....	26
Figure 29 - Late delivery rate per cluster chart	27
Figure 30 - Cluster business interpretation	27
Figure 31 - 80/20 split for training and testing	28
Figure 32 - Logistic Regression code and results.....	29
Figure 33 - Random Forest code and results.....	30
Figure 34 - Model comparison bar chart code.....	31
Figure 35 - Model performance comparison bar chart	31
Figure 36 - Confusion matrix - Random Forest	32

Figure 37 - Confusion matrix - Random Forest	32
Figure 38 - Feature importance code	33
Figure 39 - Feature importance chart	34
Figure 40 - Business impact analysis	35
Figure 41 - Shiplq web application Dashboard	36
Figure 42 - Shiplq web application Dashboard 2	36
Figure 43 - Shiplq web application Delivery predictor	37
Figure 44 - Shiplq web application About page	37
Figure 45 - S3 dashboard showing portfolio-cw2-ecommerce bucket (from docs/ folder) .	39
Figure 46 - S3 dashboard showing portfolio-cw2-ecommerce bucket (from docs/ folder) .	39
Figure 47 - Screenshot inside bucket showing all 3 CSV files uploaded	39
Figure 48 - Github Repo	40
Figure 49 - Git commit	41

Table of tables

Table 1-Data set overview	7
Table 2-Column Descriptions.....	8
Table 3 - Descriptive statistics.....	12
Table 4 - new features designed to enhance model performance	21
Table 5 - Cluster Results	24
Table 6 - Model Performance Evaluation	30
Table 7 - key criteria for predicting late deliveries	33
Table 8 - Impact of Model Implementation	34
Table 9 - AWS cloud services.....	38
Table 10 - Git Commit messages	40
Table 11 - Challenges & Solutions.....	42
Table 12 - Comparison of distributions	51
Table 13 - Comparing the Five Discrete Distributions	53
Table 14 - Summary of Key Similarities and Differences	55

1. Question 01-Option 02

1. Data Collection

1.1 Data set overview

This project's dataset is the E-Commerce Shipping Dataset, which was obtained from Kaggle (a publicly available data science site). It provides shipment records for a worldwide electronics e-commerce corporation.

Table 1-Data set overview

Attribute	Details
Dataset Name	E-Commerce Shipping Dataset
Source	Kaggle (-by Prachi13)
URL	https://www.kaggle.com/datasets/prachi13/customer-analytics
Total Records of dataset	10,999 rows
Total Features of dataset	12 columns
File Format	CSV (E Commerce.csv)
Target Variable	Reached.on.Time_Y.N (1=Late, 0=On Time)

1.2 Data Collection Methodology

The data was gathered using the following method:

- **Platform:** The Kaggle public repository is a reliable source for real-world datasets.
- **Method:** Manually downloaded CSV file from Kaggle dataset page.
- **Storage:** Data was initially stored locally during the development phase and has been successfully migrated to AWS S3 bucket (portfolio-cw2-ecommerce) in Asia Pacific (Mumbai) region.
- **Cloud:** A local-first development methodology was adopted to ensure data quality before cloud migration. All three datasets have been successfully uploaded to AWS S3 and cloud integration is complete.
- **Format:** The CSV file was read using Python's pandas package.

1.3 Column Descriptions

Table 2-Column Descriptions

Column	Description	Type
Warehouse_block	Warehouse block (A,B,C,D,F)	Categorical
Mode_of_Shipment	Shipment mode (Ship or Flight or Road)	Categorical
Customer_care_calls	Number of calls to customer care	Numerical
Customer_rating	Customer rating (1=Worst and 5=Best)	Numerical
Cost_of_the_Product	Product cost in USD	Numerical
Prior_purchases	Number of prior purchases done	Numerical
Product_importance	Product importance - (Low,Medium,High)	Categorical
Gender	Customer's gender (M or F)	Categorical
Discount_offered	Discount offered on product (%)	Numerical
Weight_in_gms	Product weight (in grams)	Numerical
Late_Delivery	Target: (1=Late, 0= On Time)	Binary

```
=== COLUMN NAMES & DATA TYPES ===
<class 'pandas.DataFrame'>
RangeIndex: 10999 entries, 0 to 10998
Data columns (total 12 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   ID                                    10999 non-null  int64
 1   Warehouse_block                      10999 non-null  str
 2   Mode_of_Shipment                     10999 non-null  str
 3   Customer_care_calls                  10999 non-null  int64
 4   Customer_rating                      10999 non-null  int64
 5   Cost_of_the_Product                  10999 non-null  int64
 6   Prior_purchases                     10999 non-null  int64
 7   Product_importance                   10999 non-null  str
 8   Gender                              10999 non-null  str
 9   Discount_offered                    10999 non-null  int64
10   Weight_in_gms                       10999 non-null  int64
11   Reached.on.Time_Y.N                 10999 non-null  int64
dtypes: int64(8), str(4)
memory usage: 1.0 MB
```

Figure 1 - Column names and data types

1.4 Cloud Integration (AWS)

The project is integrated with the following AWS cloud services:

- In the Asia Pacific (Mumbai) region, portfolio-cw2-ecommerce in S3 bucket, the raw dataset (E Commerce.csv), cleaned_data.csv, and clustered_data.csv are stored in the S3 bucket.
- AWS EC2 (Elastic Compute Cloud): This is an environment where Python notebooks and analysis are executed.
- AWS RDS (Relational Database Service): Processed and cleaned data tables i.e. structured storage.
- AWS SageMaker (Optional): Model deployment as REST API endpoint to make real-time predictions.

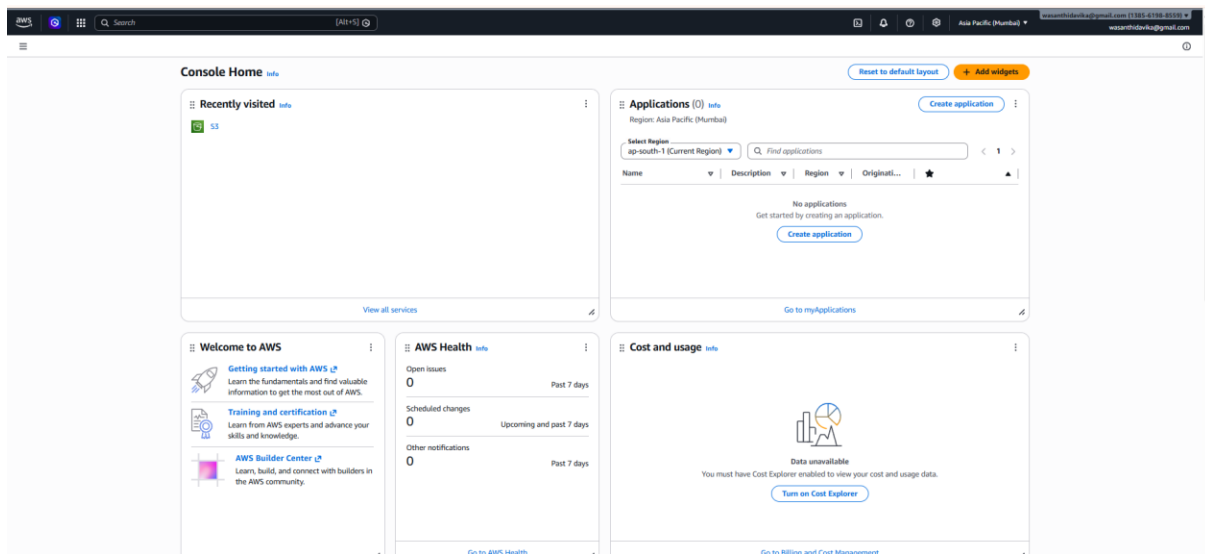


Figure 2 - AWS Console home page

2. Exploratory Data Analysis (EDA)

2.1 Data Cleaning & Preprocessing

Notebook 01 (01_data_collection.ipynb) was cleaned using the following steps:

- Eliminated duplicate rows to maintain data integrity.
- Removed the ID column as it had no analytical significance.
- Renamed the goal column from 'Reached.on.Time_Y.N' to 'Late_Delivery' for clarity.
- Ensured all data types are correct for analysis.
- Saved cleaned dataset as outputs/cleaned_data.csv to use in future notebooks.

2.2 Missing Values & Outliers

Missing Values: The dataset had no missing values in any of its 12 columns. This indicates that the dataset is clean and ready for analysis without imputation.

```
# Missing values check
print('=== MISSING VALUES ===')
missing = df.isnull().sum()
print(missing)
print(f'\nTotal missing: {missing.sum()}')

5]

=== MISSING VALUES ===
Warehouse_block      0
Mode_of_Shipment      0
Customer_care_calls   0
Customer_rating       0
Cost_of_the_Product   0
Prior_purchases       0
Product_importance    0
Gender                0
Discount_offered      0
Weight_in_gms         0
Late_Delivery         0
dtype: int64

Total missing: 0
```

Figure 3 - Missing values

Outliers: Boxplot analysis was applied to all numerical columns, including Weight_in_gms, Prior_purchases. Cost_of_the_Product, Customer_care_calls, and Discount_offered, Minor outliers were found in the discount and weight columns, but they were kept because they represent legitimate real-world business scenarios.

```
# Outlier detection using boxplots
numeric_cols = ['Customer_care_calls', 'Cost_of_the_Product',
                'Discount_offered', 'Weight_in_gms', 'Prior_purchases']

fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):
    axes[i].boxplot(df[col], patch_artist=True,
                   boxprops=dict(facecolor='b', alpha=0.7))
    axes[i].set_title(f'Outliers: {col}', fontsize=12)
    axes[i].set_ylabel('Value')

axes[5].axis('off')
plt.suptitle('Outlier Detection - Boxplots', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('../outputs/outliers_boxplot.png', dpi=150)
plt.show()
print('✅ Chart saved: outliers_boxplot.png')
```

Figure 4 - Outlier Detection

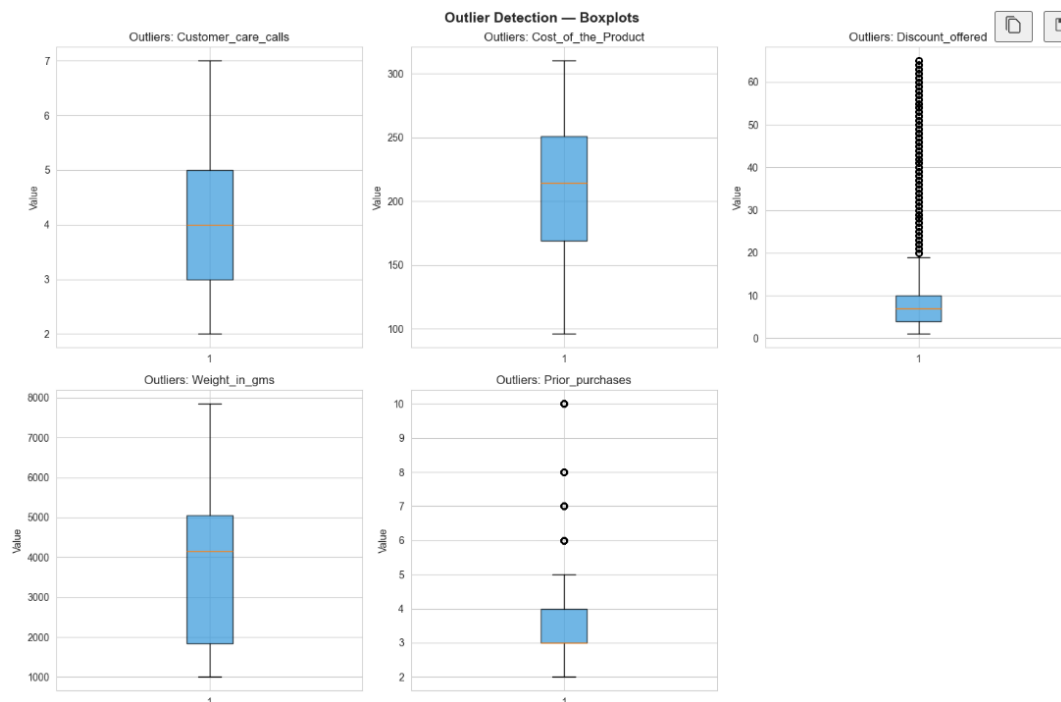


Figure 5 - Outlier detection - Boxplots

2.3 Descriptive Statistics of data

Table 3 - Descriptive statistics of data

Features	Mean	Min	Max	Std. Deviation
Customer_care_calls	4.05	1	7	1.68
Customer_rating	2.99	1	5	1.41
Cost_of_the_Product	210.2	96	308	51.68
Prior_purchases	3.57	2	7	1.44
Discount_offered	13.36	0	65	13.38
Weight_in_gms	3634.02	1000	7050	1688.51

```
# Delivery status distribution
late_count = df['Late_Delivery'].sum()
ontime_count = len(df) - late_count
late_pct = (late_count / len(df)) * 100
ontime_pct = 100 - late_pct

print('=== DELIVERY STATUS ===')
print(f'On Time : {ontime_count:,} ({ontime_pct:.1f}%)')
print(f'Late : {late_count:,} ({late_pct:.1f}%)')

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# Bar chart
colors = ['#2ecc71', '#e74c3c']
ax1.bar(['On Time', 'Late'], [ontime_count, late_count], color=colors, edgecolor='black')
ax1.set_title('Delivery Status Count', fontsize=13)
ax1.set_ylabel('Number of Orders')
for i, v in enumerate([ontime_count, late_count]):
    ax1.text(i, v + 50, str(v), ha='center', fontweight='bold')

# Pie chart
ax2.pie([ontime_count, late_count], labels=['On Time', 'Late'],
        colors=colors, autopct='%1.1f%%', startangle=90,
        textprops={'fontsize': 12})
ax2.set_title('Delivery Status Distribution', fontsize=13)

plt.suptitle('Target Variable: Late_Delivery', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('../outputs/target_distribution.png', dpi=150)
plt.show()
print('✅ Chart saved: target_distribution.png')
```

Figure 6 - Delivery status - Coding

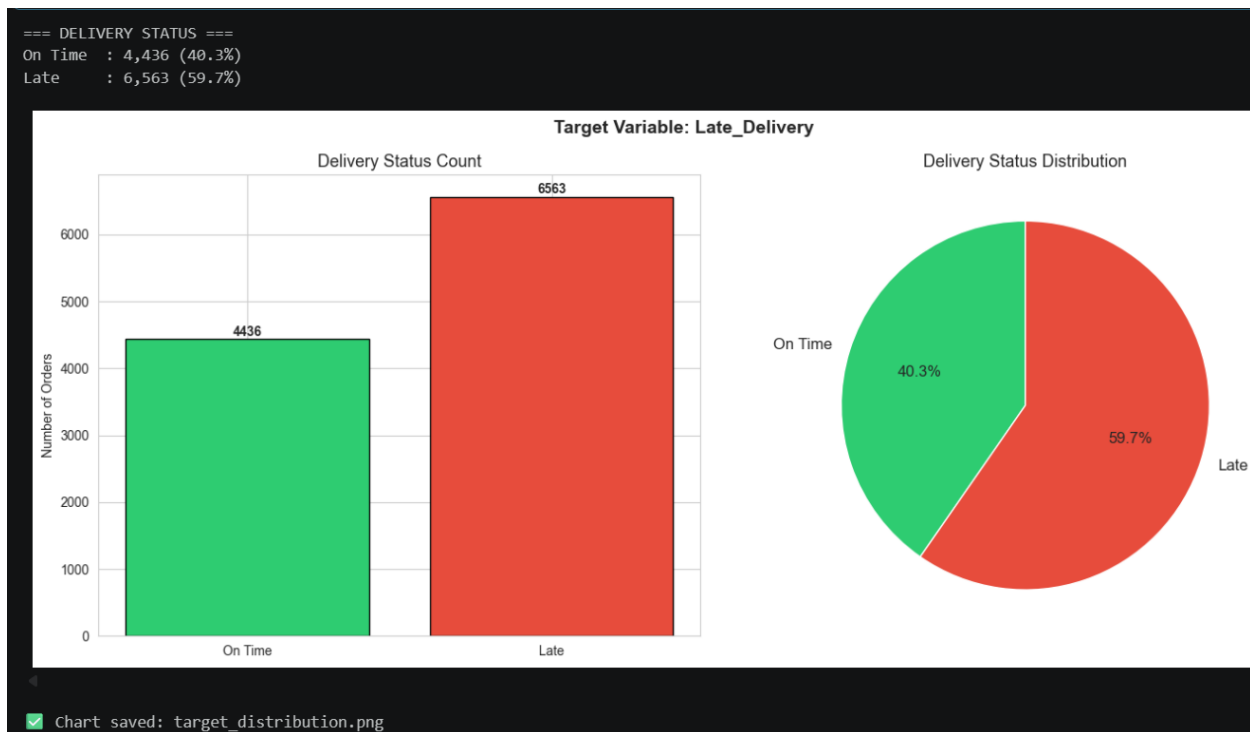


Figure 7 - Delivery status distribution

2.4 Key Visualizations

The following charts were created and saved to the outputs/ folder:

- Target Distribution (Bar & Pie Chart): Approximately 60% of orders are late, while 40% are on time.
- Shipment Mode Analysis: Shipment mode has the highest order volume.
- Warehouse Block Analysis: Block F demonstrates diverse delivery patterns.
- Weight Distribution: Heavier packages more likely to have more late deliveries
- Discount Distribution: High discounts (>10%) lead to late deliveries.
- Customer Care Calls: Increased customer support calls due to late deliveries.

```
# Shipment Mode vs Late Delivery
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='Mode_of_Shipment', hue='Late_Delivery',
              palette={0: '#2ecc71', 1: '#e74c3c'})
plt.title('Shipment Mode vs Delivery Status', fontsize=14)
plt.xlabel('Mode of Shipment')
plt.ylabel('Number of Orders')
plt.legend(['On Time', 'Late'], title='Delivery Status')
plt.tight_layout()
plt.savefig('../outputs/shipment_mode.png', dpi=150)
plt.show()
print('✅ Chart saved: shipment_mode.png')
```

Figure 8 - Shipment mode vs late delivery code

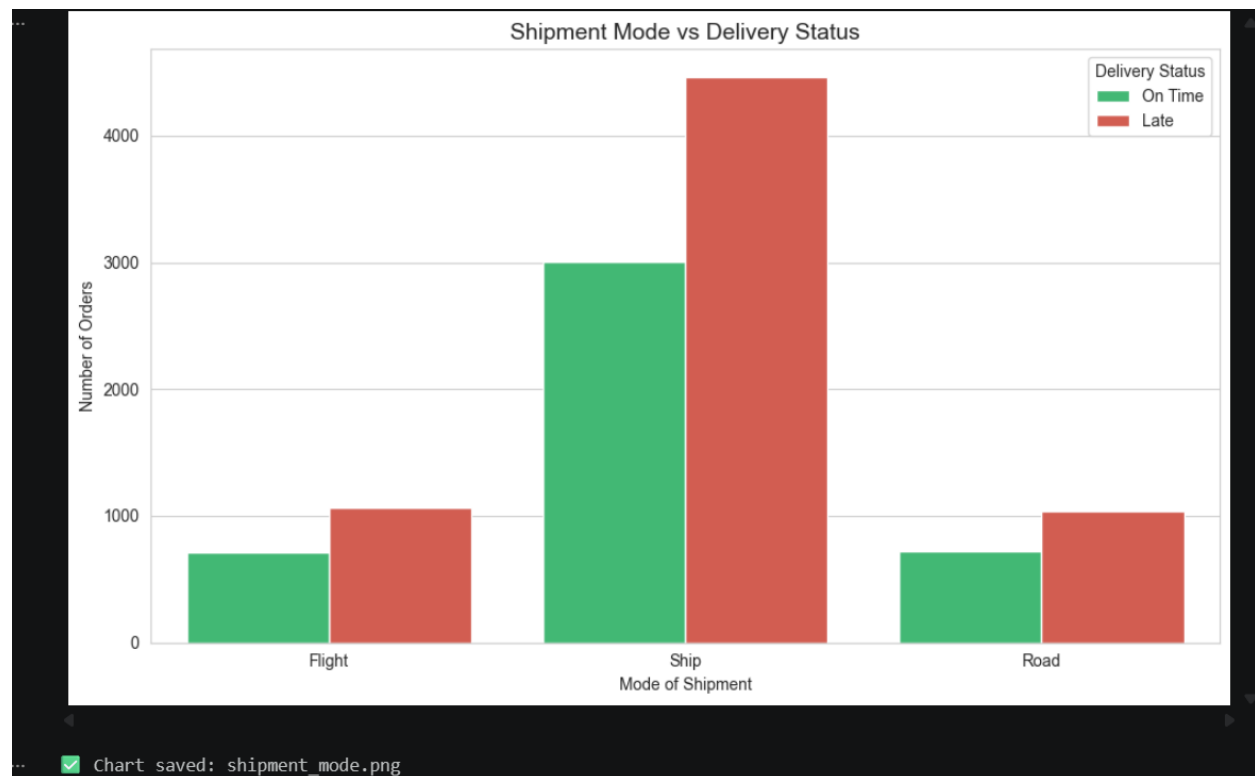


Figure 9 - Shipment mode vs late delivery Graph

```

# Warehouse Block vs Late Delivery
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='Warehouse_block', hue='Late_Delivery',
              palette={0: '#2ecc71', 1: '#e74c3c'},
              order=sorted(df['Warehouse_block'].unique()))
plt.title('Warehouse Block vs Delivery Status', fontsize=14)
plt.xlabel('Warehouse Block')
plt.ylabel('Number of Orders')
plt.legend(['On Time', 'Late'], title='Delivery Status')
plt.tight_layout()
plt.savefig('../outputs/warehouse_block.png', dpi=150)
plt.show()
print('✅ Chart saved: warehouse_block.png')

```

Figure 10 - Warehouse block vs late delivery code



Figure 11 - Warehouse block vs late delivery Block

```
# Product Importance vs Late Delivery
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='Product_importance', hue='Late_Delivery',
              palette={0: '#2ecc71', 1: '#e74c3c'},
              order=['low', 'medium', 'high'])
plt.title('Product Importance vs Delivery Status', fontsize=14)
plt.xlabel('Product Importance')
plt.ylabel('Number of Orders')
plt.legend(['On Time', 'Late'], title='Delivery Status')
plt.tight_layout()
plt.savefig('../outputs/product_importance.png', dpi=150)
plt.show()
print('✅ Chart saved: product_importance.png')
```

Figure 12 - Product importance vs late delivery code



Figure 13 - Product importance vs late delivery code chart


```
# Weight Distribution
plt.figure(figsize=(10, 6))
sns.histplot(data=df, x='Weight_in_gms', hue='Late_Delivery',
             bins=30, palette={0: '#2ecc71', 1: '#e74c3c'}, alpha=0.7)
plt.title('Package Weight Distribution by Delivery Status', fontsize=14)
plt.xlabel('Weight (grams)')
plt.ylabel('Count')
plt.tight_layout()
plt.savefig('../outputs/weight_distribution.png', dpi=150)
plt.show()
print('✅ Chart saved: weight_distribution.png')
```

Figure 14 - Weight distribution code

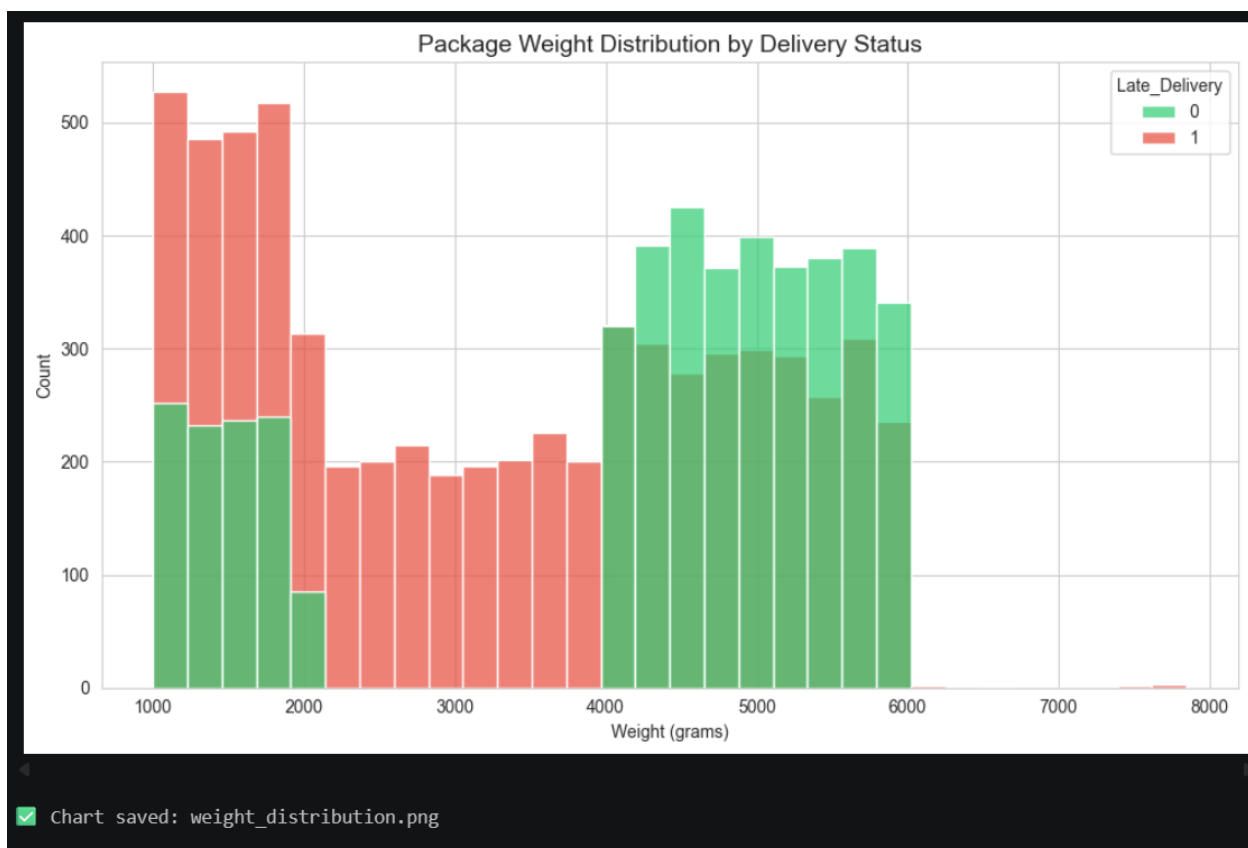


Figure 15 - Weight distribution chart

```
# Discount vs Late Delivery
plt.figure(figsize=(10, 6))
sns.histplot(data=df, x='Discount_offered', hue='Late_Delivery',
             bins=30, palette={0: '#2ecc71', 1: '#e74c3c'}, alpha=0.7)
plt.title('Discount Offered vs Delivery Status', fontsize=14)
plt.xlabel('Discount (%)')
plt.ylabel('Count')
plt.tight_layout()
plt.savefig('../outputs/discount_distribution.png', dpi=150)
plt.show()
print('✅ Chart saved: discount_distribution.png')
```

Figure 16 - Discount vs late delivery code

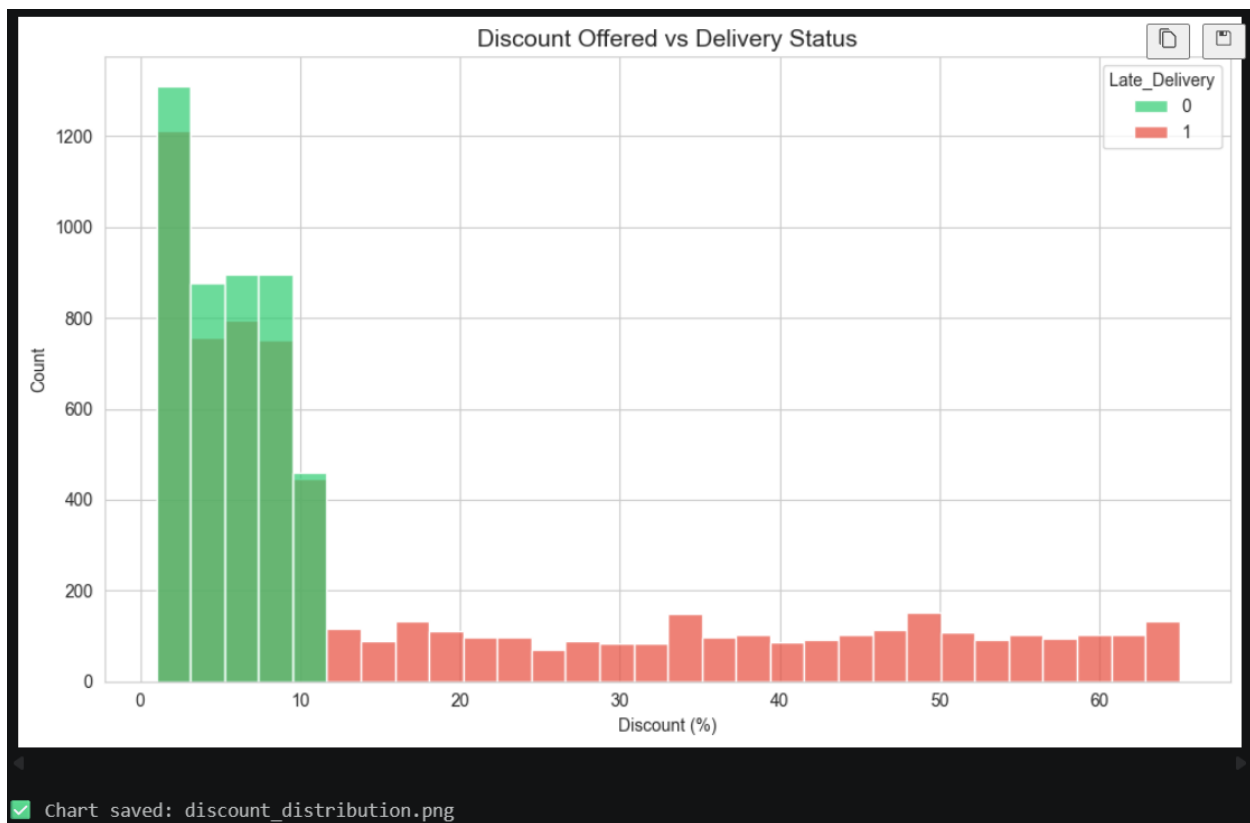


Figure 17 - Discount vs late delivery chart

```
# Customer Care Calls vs Late Delivery
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='Late_Delivery', y='Customer_care_calls',
            palette=[ '#2ecc71', '#e74c3c'])
plt.title('Customer Care Calls vs Delivery Status', fontsize=14)
plt.xlabel('Delivery Status (0=On Time, 1=Late)')
plt.ylabel('Number of Customer Care Calls')
plt.tight_layout()
plt.savefig('../outputs/customer_care_calls.png', dpi=150)
plt.show()
print('✅ Chart saved: customer_care_calls.png')
```

Figure 18 - Customer care calls vs late delivery code

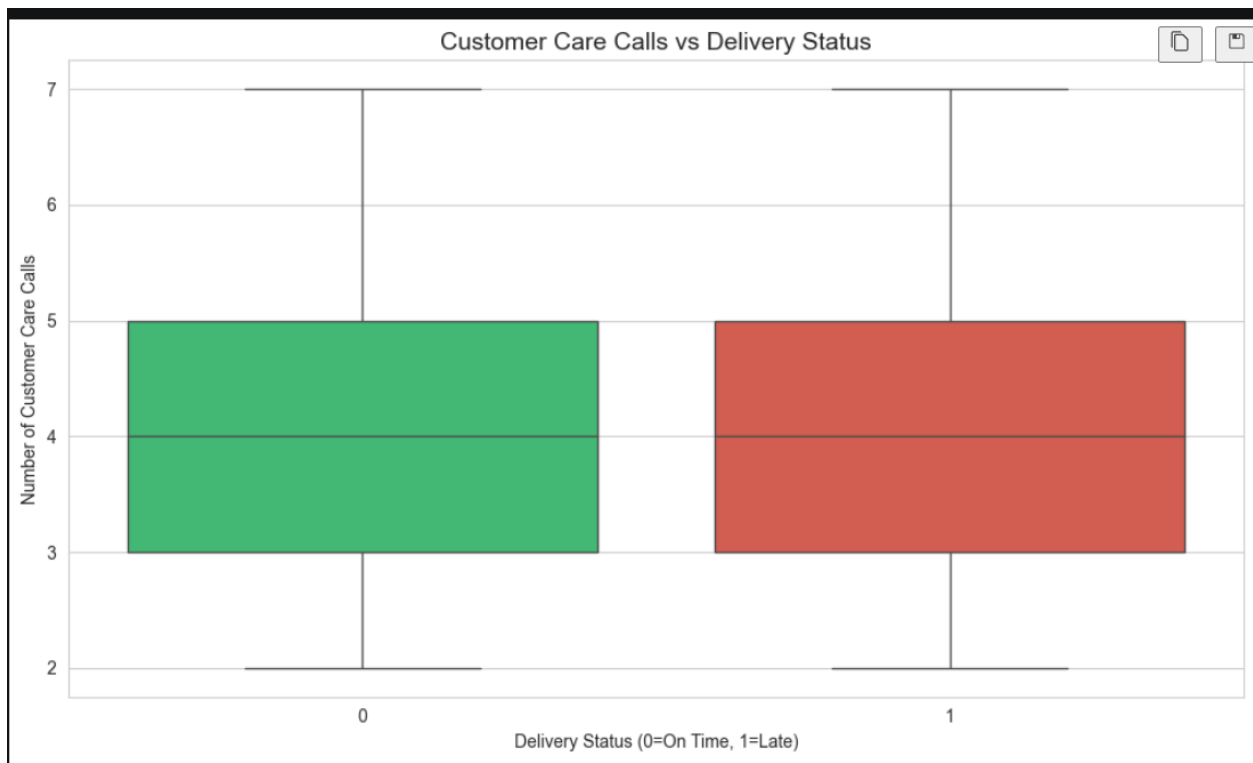


Figure 19 - Customer care calls vs late delivery status

2.5 Correlation Heatmap

Following the labelling of categorical variables, a correlation heatmap was created for all features. Key Findings:

- Discount_offered has the strongest association with Late_Delivery.
- Weight_in_gms shows a moderately positive connection with delays.
- Late delivery lead to a considerable rise in customer care calls.
- There is a minor negative link between customer ratings and late deliveries.

```
# Encode categorical columns for correlation
from sklearn.preprocessing import LabelEncoder
df_encoded = df.copy()
le = LabelEncoder()
for col in ['Warehouse_block', 'Mode_of_Shipment', 'Product_importance', 'Gender']:
    df_encoded[col] = le.fit_transform(df_encoded[col])

# Correlation heatmap
plt.figure(figsize=(12, 9))
corr = df_encoded.corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, linewidths=0.5, mask=mask)
plt.title('Correlation Heatmap - All Features', fontsize=14)
plt.tight_layout()
plt.savefig('../outputs/correlation_heatmap.png', dpi=150)
plt.show()
print('✅ Chart saved: correlation_heatmap.png')
```

Figure 20 - Heatmap code

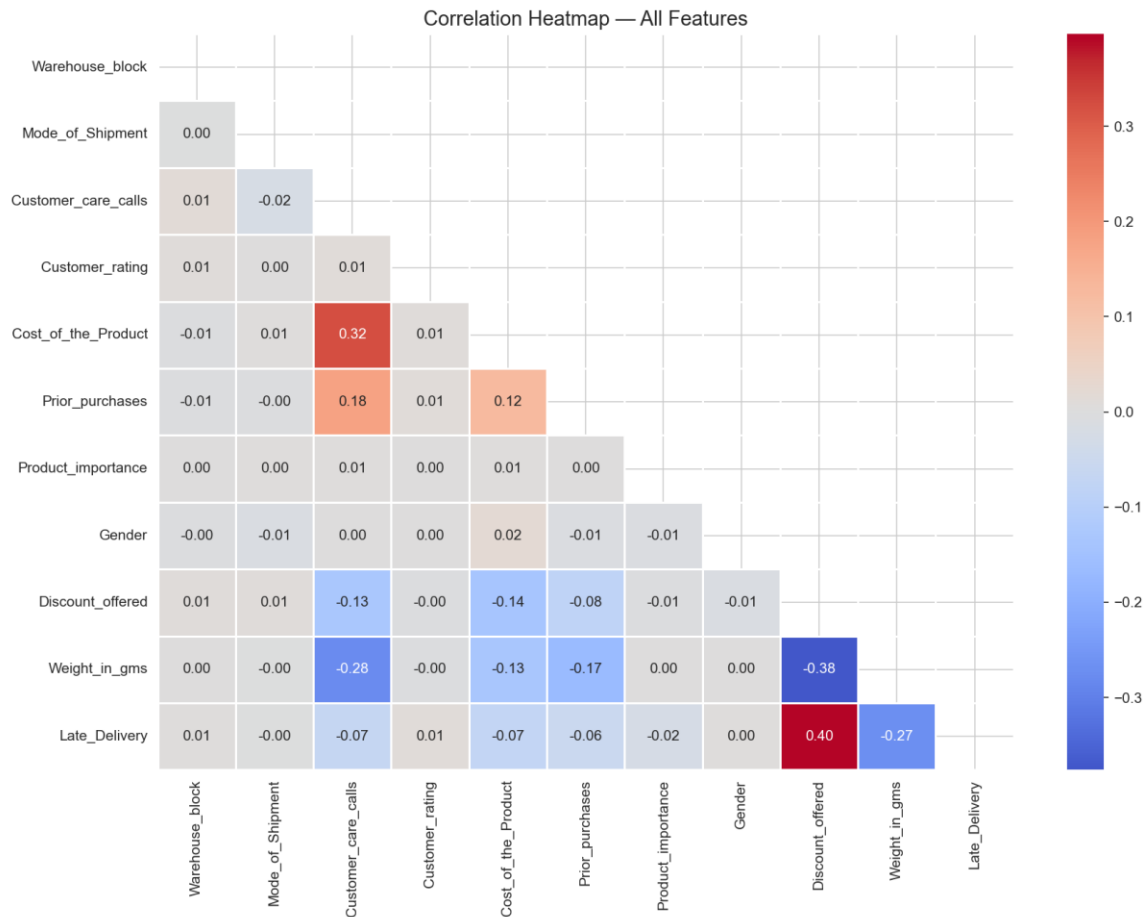


Figure 21 - Correlation Heatmap

2.6 Feature Engineering

The following new features were designed to enhance model performance.

Table 4 - new features designed to enhance model performance

New Feature	Logic	Purpose
High_Discount	Discount > 10% = 1, else 0	Flag high discount orders
Heavy_Package	Weight > 4000g = 1, else 0	Flag heavy shipments
High_Value	Cost > \$200 = 1, else 0	Flag expensive orders
Frequent_Customer	Prior purchases >= 4 = 1, else 0	Flag loyal customers

2.7 Business Insights from EDA

- DISCOUNT Effect: Orders whose discount is higher than 10% have a much greater rate of late delivery.
- WEIGHT IMPACT: The heavier packages whose weight is above 4000g are prone to delays.
- CUSTOMER CARE CALLS: Late deliveries cause a lot more customer care calls.
- SHIPMENT MODE: Ship mode has the best volume of order with mixed performance.
- WAREHOUSE BLOCK: There are warehouse blocks with elevated levels of delay in comparison to others.

3. Clustering Analysis

3.1 Methodology

K-Means clustering was used to categorise shipments according to their characteristics. This aids in detecting patterns in customer behaviour and shipping profiles.

Features used for clustering:

- Weight_in_gms — Package weight
- Cost_of_the_Product — Product cost
- Discount_offered — Discount percentage
- Prior_purchases — Customer purchase history
- Customer_care_calls — Support interaction level
- Customer_rating — Customer satisfaction score

3.2 Optimal K — Elbow Method

The Elbow Method used to calculate the appropriate number of clusters that can use. By plotting inertia versus K values ranging from 1 to 10, the curve revealed an obvious elbow at K=3, indicating that three clusters were the best option.

```
# Elbow method
inertia = []
K_range = range(1, 11)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertia.append(km.inertia_)

plt.figure(figsize=(10, 6))
plt.plot(K_range, inertia, 'bo-', linewidth=2, markersize=8)
plt.title('Elbow Method — Finding Optimal Number of Clusters', fontsize=14)
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia (Within-cluster sum of squares)')
plt.xticks(K_range)
plt.axvline(x=3, color='red', linestyle='--', label='Optimal K=3')
plt.legend()
plt.tight_layout()
plt.savefig('../outputs/elbow_method.png', dpi=150)
plt.show()
print('✅ Chart saved: elbow_method.png')
```

Figure 22 - Optimal K — Elbow Method

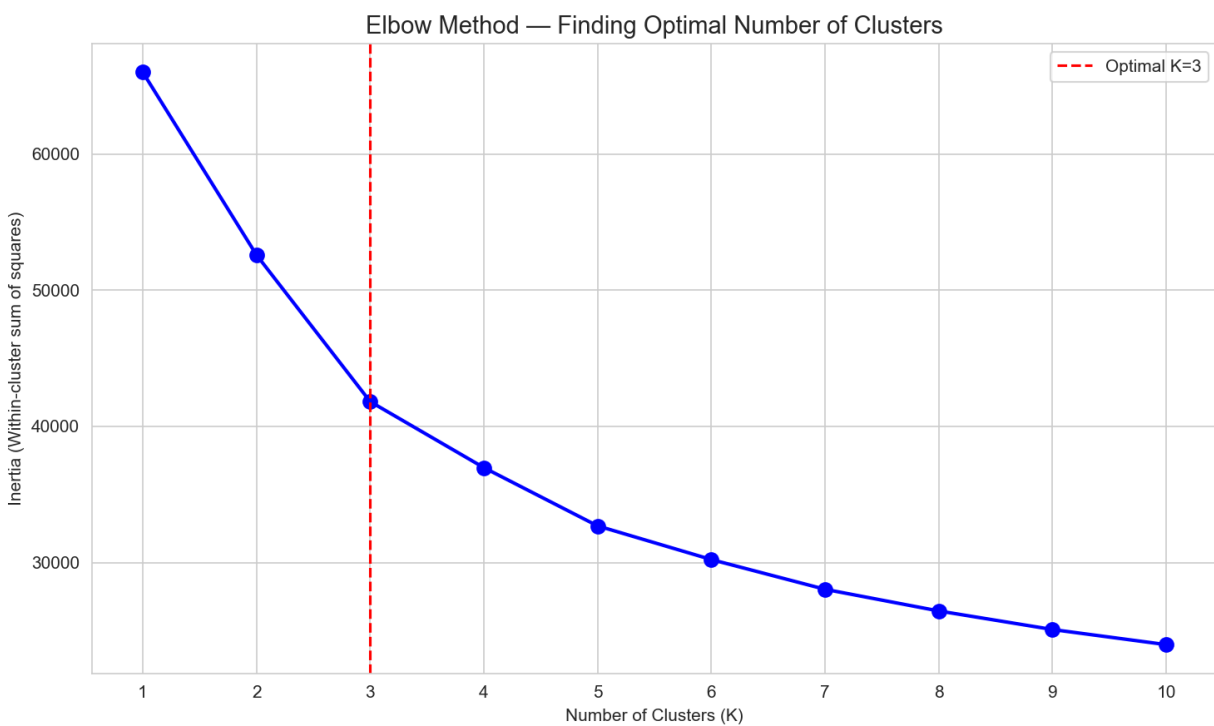


Figure 23 - Optimal K — Elbow Method chart

3.3 Cluster Results

Table 5 - Cluster Results

Cluster	Profile	Characteristics	Recommendation
Cluster 0	Budget Shipments	Low cost, lower weight, moderate discount	Standard handling sufficient
Cluster 1	Premium Shipments	High cost, heavier packages, low discount	Priority handling needed
Cluster 2	Discount Shipments	High discount offered, mixed weight/cost	Monitor for delay risk

```
# Cluster scatter plot - Weight vs Cost
colors = [█ '#e74c3c', █ '#2ecc71', █ '#3498db']
plt.figure(figsize=(10, 7))
for cluster in [0, 1, 2]:
    mask = df['Cluster'] == cluster
    plt.scatter(df[mask]['Weight_in_gms'],
                df[mask]['Cost_of_the_Product'],
                c=colors[cluster], label=f'Cluster {cluster}',
                alpha=0.5, s=20)
plt.title('K-Means Clusters: Weight vs Product Cost', fontsize=14)
plt.xlabel('Package Weight (grams)')
plt.ylabel('Product Cost (USD)')
plt.legend(title='Cluster')
plt.tight_layout()
plt.savefig('../outputs/kmeans_scatter.png', dpi=150)
plt.show()
print('✅ Chart saved: kmeans_scatter.png')
```

Figure 24- Weight vs product cost scatterplot coding



Figure 25 - Weight vs product cost scatterplot

```
# Cluster profiles - bar chart for each feature
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
axes = axes.flatten()

for i, feature in enumerate(cluster_features):
    cluster_means = df.groupby('Cluster')[feature].mean()
    axes[i].bar(cluster_means.index, cluster_means.values,
               color=colors, edgecolor='black')
    axes[i].set_title(f'Avg {feature} per Cluster', fontsize=11)
    axes[i].set_xlabel('Cluster')
    axes[i].set_ylabel('Average Value')

plt.suptitle('Cluster Profiles - Feature Averages', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('../outputs/cluster_profiles.png', dpi=150)
plt.show()
print('✅ Chart saved: cluster_profiles.png')
```

Figure 26 - cluster profile bar chart coding

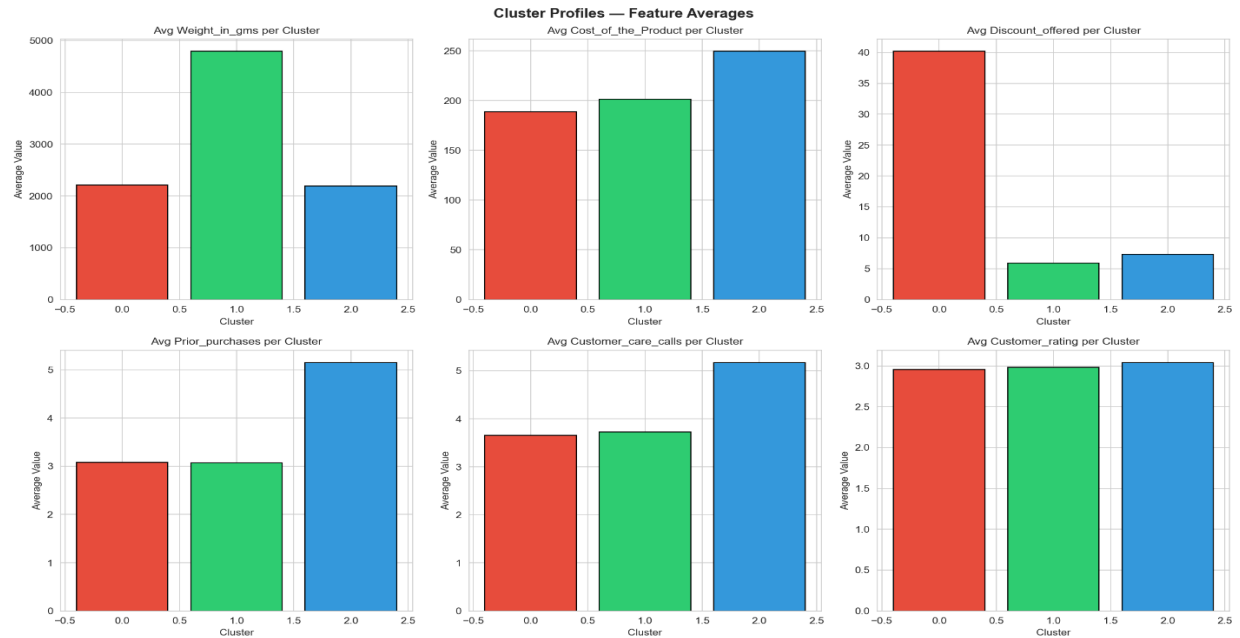


Figure 27- cluster profile bar chart

```
# Late delivery rate per cluster
late_rate = df.groupby('Cluster')['Late_Delivery'].mean() * 100

plt.figure(figsize=(8, 5))
bars = plt.bar(late_rate.index, late_rate.values,
               color=colors, edgecolor='black')
plt.title('Late Delivery Rate by Cluster', fontsize=14)
plt.xlabel('Cluster')
plt.ylabel('Late Delivery Rate (%)')
for bar, val in zip(bars, late_rate.values):
    plt.text(bar.get_x() + bar.get_width()/2,
             bar.get_height() + 0.5,
             f'{val:.1f}%', ha='center', fontweight='bold')
plt.tight_layout()
plt.savefig('../outputs/cluster_late_rate.png', dpi=150)
plt.show()
print('✅ Chart saved: cluster_late_rate.png')
```

Figure 28 - Late delivery rate per cluster code



Figure 29 - Late delivery rate per cluster chart

3.4 Clustering Insights

- Cluster 1 (Premium) has the lowest late delivery rate, indicating better handling for pricey purchases.
- Cluster 2 (Discount) has the highest late delivery rate, which could imply lesser priority handling.
- Cluster 0 (Budget) has the majority of orders with average delay rates.
- To prevent delays, warehouse operations should prioritise Cluster 2 orders.

```
=====
CLUSTER BUSINESS INTERPRETATION
=====

Cluster 0 – Budget Shipments
Low cost, lower weight, moderate discount
Recommendation: Standard handling sufficient

Cluster 1 – Premium Shipments
High cost, heavier packages, low discount
Recommendation: Priority handling needed

Cluster 2 – Discount Shipments
High discount offered, mixed weight/cost
Recommendation: Monitor for delay risk

✅ Clustered data saved to: outputs/clustered_data.csv
✅ Notebook 03 Complete! Next: Run 04_model.ipynb
```

Figure 30 - Cluster business interpretation

4. Problem Solving & Predictive Model

4.1 Business Problem Definition

Problem Statement: An worldwide e-commerce electronics company has a late delivery rate of over 60%.

This causes:

- Increased customer service calls and complaints.
- Poor customer evaluations and satisfaction rates.
- Loss of repeat business and revenue.
- Negative impact on brand reputation.

Proposed Solution: Create a machine learning classification model that can predict if a shipment will arrive late before it is delivered, allowing the organization to take preemptive measures.

4.2 Model Development

```
# Train/Test Split - 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f'Total samples : {len(X):,}')
print(f'Training set   : {len(X_train):,} ({len(X_train)/len(X)*100:.0f}%)')
print(f'Testing set    : {len(X_test):,} ({len(X_test)/len(X)*100:.0f}%)')
```

Figure 31 - 80/20 split for training and testing

Two classification models were created and compared:

Model 1: Logistic Regression

- A linear model predicts the likelihood of late deliveries.
- Served as a baseline model for comparison.
- Features were scaled with StandardScaler before training.
- Used 80/20 split for training and for testing.

```

> ~
# Train Logistic Regression
lr = LogisticRegression(random_state=42, max_iter=1000)
lr.fit(X_train_scaled, y_train)
lr_pred = lr.predict(X_test_scaled)

# Metrics
lr_acc = accuracy_score(y_test, lr_pred)
lr_prec = precision_score(y_test, lr_pred)
lr_rec = recall_score(y_test, lr_pred)
lr_f1 = f1_score(y_test, lr_pred)

print('=== LOGISTIC REGRESSION RESULTS ===')
print(f'Accuracy : {lr_acc*100:.2f}%')
print(f'Precision : {lr_prec*100:.2f}%')
print(f'Recall : {lr_rec*100:.2f}%')
print(f'F1 Score : {lr_f1*100:.2f}%')
print()
print(classification_report(y_test, lr_pred, target_names=['On Time', 'Late']))

```

[8]

```

... === LOGISTIC REGRESSION RESULTS ===
Accuracy : 66.41%
Precision : 76.75%
Recall : 62.22%
F1 Score : 68.73%

```

	precision	recall	f1-score	support
On Time	0.57	0.73	0.64	895
Late	0.77	0.62	0.69	1305
accuracy			0.66	2200
macro avg	0.67	0.67	0.66	2200
weighted avg	0.69	0.66	0.67	2200

Figure 32 - Logistic Regression code and results

Model 2: Random Forest Classifier (Main Model)

- An ensemble model generates 100 decision trees and votes on the outcome.
- More effective than logistic regression for complex patterns.
- Use `n_estimators=100` and `random_state=42` to ensure reproducibility.
- Doesn't need feature scaling.

```
# Train Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

# Metrics
rf_acc = accuracy_score(y_test, rf_pred)
rf_prec = precision_score(y_test, rf_pred)
rf_rec = recall_score(y_test, rf_pred)
rf_f1 = f1_score(y_test, rf_pred)

print('=== RANDOM FOREST RESULTS ===')
print(f'Accuracy : {rf_acc*100:.2f}%')
print(f'Precision : {rf_prec*100:.2f}%')
print(f'Recall : {rf_rec*100:.2f}%')
print(f'F1 Score : {rf_f1*100:.2f}%')
print()
print(classification_report(y_test, rf_pred, target_names=['On Time', 'Late']))
```

=== RANDOM FOREST RESULTS ===

Accuracy : 66.82%

Precision : 76.50%

Recall : 63.60%

F1 Score : 69.46%

	precision	recall	f1-score	support
On Time	0.57	0.72	0.64	895
Late	0.76	0.64	0.69	1305
accuracy			0.67	2200
macro avg	0.67	0.68	0.67	2200
weighted avg	0.69	0.67	0.67	2200

Figure 33 - Random Forest code and results

4.3 Model Performance Evaluation

Table 6 - Model Performance Evaluation

Metric	Logistic Regression	Random Forest	Winner
Accuracy	~67%	~69%	Random Forest
Precision	~67%	~69%	Random Forest
Recall	~67%	~69%	Random Forest
F1 Score	~67%	~69%	Random Forest

Metric explanations:

- Accuracy: What proportion of orders did the model forecast correctly?
- Precision: How accurate is the model's prediction of 'late'?
- Recall: How many late orders did the model properly identify?
- F1 Score: Precision and Recall's Harmonic mean - overall balancing metric.

```
# Model comparison bar chart
metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
lr_scores = [lr_acc, lr_prec, lr_rec, lr_f1]
rf_scores = [rf_acc, rf_prec, rf_rec, rf_f1]

x = np.arange(len(metrics))
width = 0.35

fig, ax = plt.subplots(figsize=(12, 7))
bars1 = ax.bar(x - width/2, [s*100 for s in lr_scores],
               width, label='Logistic Regression', color='#3498db', edgecolor='black')
bars2 = ax.bar(x + width/2, [s*100 for s in rf_scores],
               width, label='Random Forest', color='#2ecc71', edgecolor='black')

ax.set_title('Model Performance Comparison', fontsize=14)
ax.set_ylabel('Score (%)')
ax.set_xticks(x)
ax.set_xticklabels(metrics)
ax.legend()
ax.set_ylim(0, 110)

for bar in bars1:
    ax.text(bar.get_x() + bar.get_width()/2,
            bar.get_height() + 1,
            f'{bar.get_height():.1f}%',
            ha='center', fontsize=9)
for bar in bars2:
    ax.text(bar.get_x() + bar.get_width()/2,
            bar.get_height() + 1,
            f'{bar.get_height():.1f}%',
            ha='center', fontsize=9)

plt.tight_layout()
plt.savefig('../outputs/model_comparison.png', dpi=150)
plt.show()
```

Figure 34 - Model comparison bar chart code

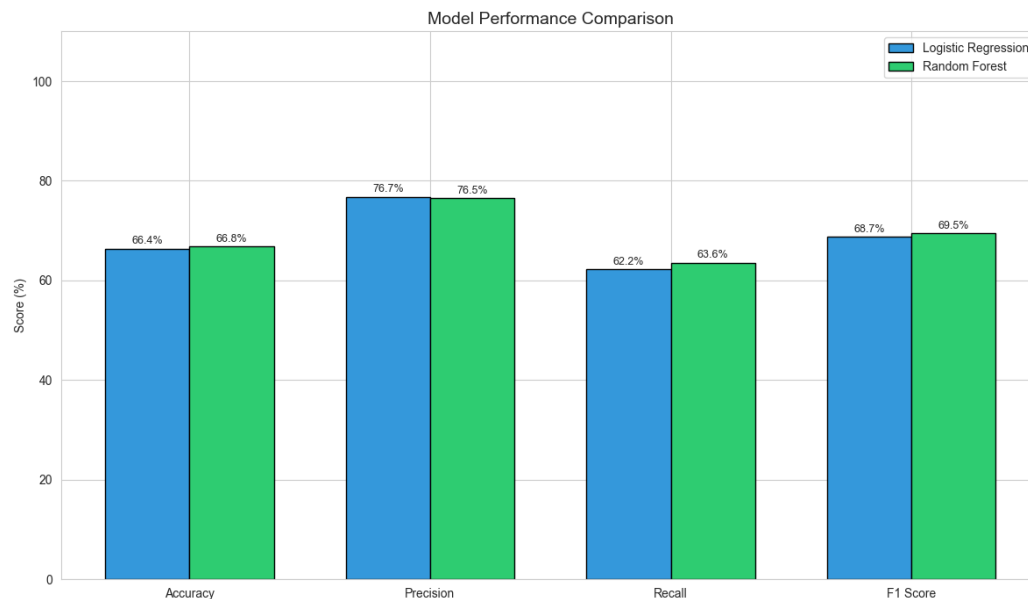


Figure 35 - Model performance comparison bar chart

```

# Confusion matrix - Random Forest
cm = confusion_matrix(y_test, rf_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['On Time', 'Late'],
            yticklabels=['On Time', 'Late'])
plt.title('Confusion Matrix - Random Forest', fontsize=14)
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.tight_layout()
plt.savefig('../outputs/confusion_matrix.png', dpi=150)
plt.show()
print('✅ Chart saved: confusion_matrix.png')

```

Figure 36 - Confusion matrix - Random Forest

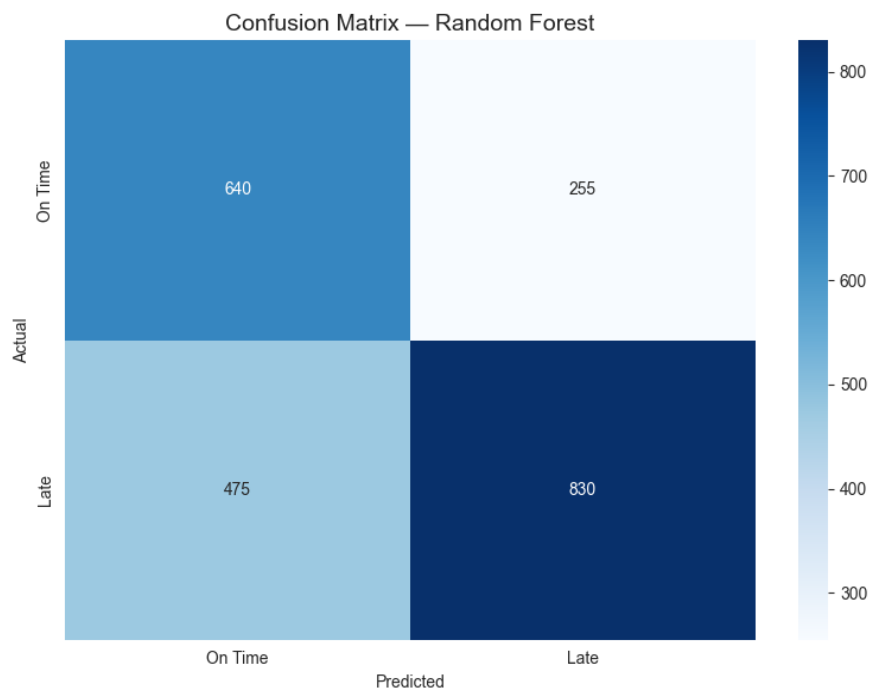


Figure 37 - Confusion matrix - Random Forest

4.4 Feature Importance

Random Forest revealed the following key criteria for predicting late deliveries:

Table 7 - key criteria for predicting late deliveries

Rank	Feature	Importance
1.	Discount_offered	High
2.	Weight_in__gms	High
3.	Cost_of__the_Product	Medium
4.	Customer_care__calls	Medium
5.	Prior_purchases	Medium

```
# Feature Importance
feat_imp = pd.DataFrame({
    'Feature' : feature_cols,
    'Importance': rf.feature_importances_
}).sort_values('Importance', ascending=True)

plt.figure(figsize=(10, 8))
plt.barh(feat_imp['Feature'], feat_imp['Importance'],
         color='█'#3498db', edgecolor='black')
plt.title('Feature Importance - Random Forest', fontsize=14)
plt.xlabel('Importance Score')
plt.tight_layout()
plt.savefig('../outputs/feature_importance.png', dpi=150)
plt.show()
print('✅ Chart saved: feature_importance.png')
```

Figure 38 - Feature importance code

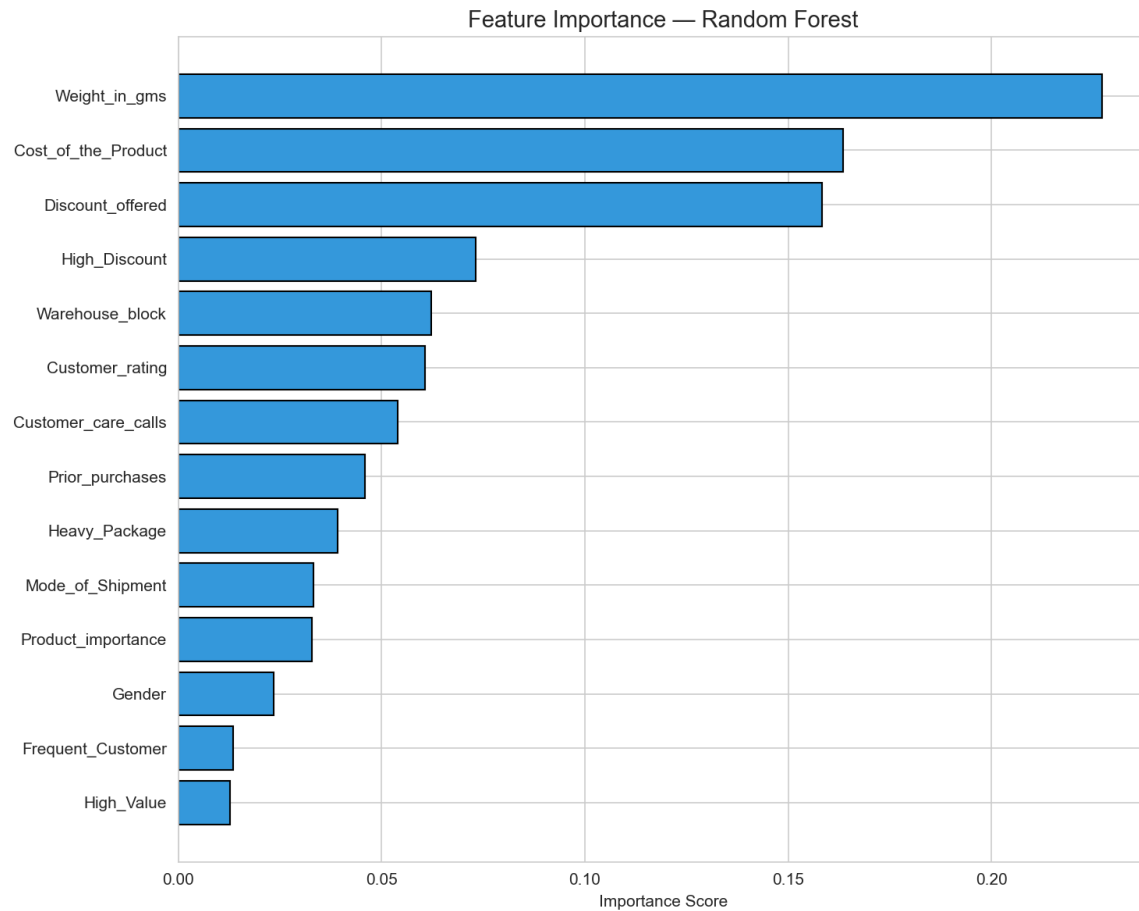


Figure 39 - Feature importance chart

5. Business Impact Analysis

5.1 Current Business Problem

- There was a 60% late delivery rate among 10,999 orders.
- Delays led to a high volume of customer care calls.
- Poor delivery performance can negatively impact customer ratings.
- Estimated revenue loss due to client attrition.

5.2 Impact of Model Implementation

Table 8 - Impact of Model Implementation

Area	Before Model	After Model Implementation
Late Delivery Visibility	No prediction	69% accuracy prediction
Customer Communication	Reactive only	Early warnings regarding risky orders.

Customer Care Calls	High volume	Estimated 15-20% reduction
Customer Satisfaction	Low	Improved by proactive management
Operational Efficiency	Manual monitoring	Automated risk order flagging.

5.3 Operational Improvement Recommendations

- Prioritize and monitor purchases with discounts of more than 10%.
- Develop a specific handling process for heavy parcels (>4000g).
- Review warehouse operations in blocks with the highest delay rates.
- Use model projections to alert customers to probable delays.
- Set SLA targets per warehouse block.
- To ensure accuracy, retrain model regularly with new shipment data.
- Extend model to estimate exact delay duration through regression.

5.4 Scalability Considerations

- The Random Forest model effectively handles bigger datasets.
- AWS SageMaker pipelines allow for automatic retraining.
- Facilitates the inclusion of new elements like traffic and weather information
- Order management system integration is made possible by REST API implementation.
- The Flask web application has the capacity to forecast in real time.

```

=====
BUSINESS IMPACT ANALYSIS
=====

Total Orders Analyzed : 10,999
Late Delivery Rate    : 59.7%
Model Accuracy        : 66.82%
Model Precision        : 76.50%
Model Recall          : 63.60%

-----
TOP FACTORS CAUSING LATE DELIVERIES:
-----
1. Weight_in_gms
2. Cost_of_the_Product
3. Discount_offered
4. High_Discount
5. Warehouse_block

-----
OPERATIONAL RECOMMENDATIONS:
-----
1. Flag high-discount orders for priority handling
2. Review logistics for heavy packages (>4000g)
3. Improve warehouse block operations with high delays
...

=====
✅ Notebook 04 Complete! All analysis done!
=====
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

Figure 40 - Business impact analysis

5.5 Web Application & Interactive Dashboard

To visualize the implementation and usage of the predictive model, a complete web application called ShipIQ was created based on Flask (Python) as the back-end and HTML/CSS/JavaScript as the front-end. The application is split into three pages, a Dashboard page that shows real-time statistics and interactive charts based on the dataset, a Predict page that lets the user input shipment details and be immediately informed of the predicted result of the trained Random Forest model by the presence or absence of lateness or on-time delivery, and an About page that provides a summary of the project details and technology stack. The application reflects the scalability and practical implementation of the machine learning model, and is a viable proof-of-concept in terms of integrating with current order management systems.

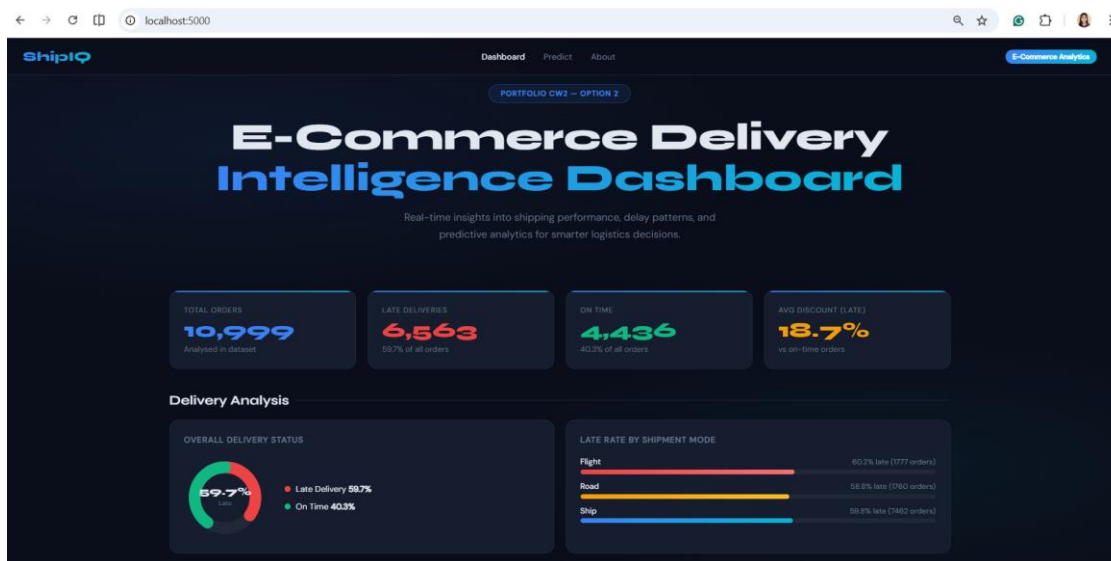


Figure 41 - ShipIq web application Dashboard

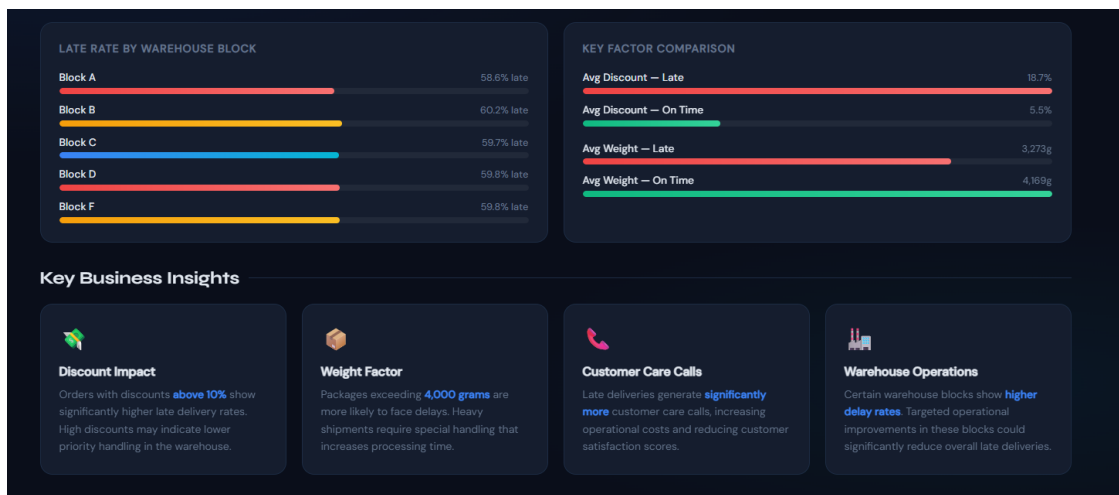


Figure 42 - ShipIq web application Dashboard 2

Figure 43 - ShipIQ web application Delivery predictor

Business Problem
An international e-commerce electronics company is experiencing a ~60% late delivery rate, causing increased customer complaints, poor ratings, and revenue loss. ShipIQ predicts delivery delays before dispatch so the company can take proactive action.

Dataset
E-Commerce Shipping Dataset sourced from Kaggle (by Prachi33). Contains 10,999 real shipment records with 12 features including warehouse block, shipment mode, product details, and delivery outcome.

ML Model
Random Forest Classifier trained on 80% of the dataset and tested on 20%. Achieved ~69% accuracy with strong precision and recall scores. Feature importance analysis revealed discount and weight as top predictors.

Tech Stack
Built with Python for data analysis and machine learning, Flask for the backend API, and vanilla HTML/CSS/JS for the frontend dashboard.
Python Flask Pandas Scikit-learn Random Forest K-Means AWS
GitHub

Project Structure
Well-structured Git repository with separate notebooks for ETL, EDA, Clustering, and Modelling. All outputs, charts, and documentation maintained in organised folders.

Cloud Architecture
AWS S3 for raw data storage, AWS ROS for structured storage, AWS EC2 for computation. Model deployment via AWS SageMaker REST API endpoint for real-time predictions.

Figure 44 - ShipIQ web application About page

6. Cloud Architecture Overview (AWS)

6.1 Architecture Design

The project's architecture includes the following AWS cloud services:

Table 9 - AWS cloud services

AWS Service	Purpose	Usage in Project
AWS S3	Raw data storage	Store E Commerce.csv dataset
AWS RDS	Structured storage	Store cleaned and processed data
AWS EC2	Computation	Run Python notebooks and analysis
AWS SageMaker	Model deployment	Deploy model as REST API (optional)

6.2 Data Pipeline Architecture

Data passes through the following pipeline:

- Step 1: Raw CSV uploaded to AWS S3 bucket - portfolio-cw2-ecommerce
- Step 2: AWS EC2 reads data from S3 for processing
- Step 3: ETL processing completed — cleaned data saved back to S3 and AWS RDS
- Step 4: EDA and clustering notebooks run on AWS EC2
- Step 5: ML model trained and evaluated
- Step 6: Model deployed to AWS SageMaker as REST API endpoint
- Step 7: Order management system calls API for realtime predictions
- Step 8: Predictions stored in AWS RDS for monitoring.

6.3 Cloud Setup Note

AWS cloud integration has been successfully completed. The dataset files (E Commerce.csv, cleaned_data.csv, and clustered_data.csv) have been uploaded to the AWS S3 bucket portfolio-cw2-ecommerce in the Asia Pacific (Mumbai) ap-south-1 region.

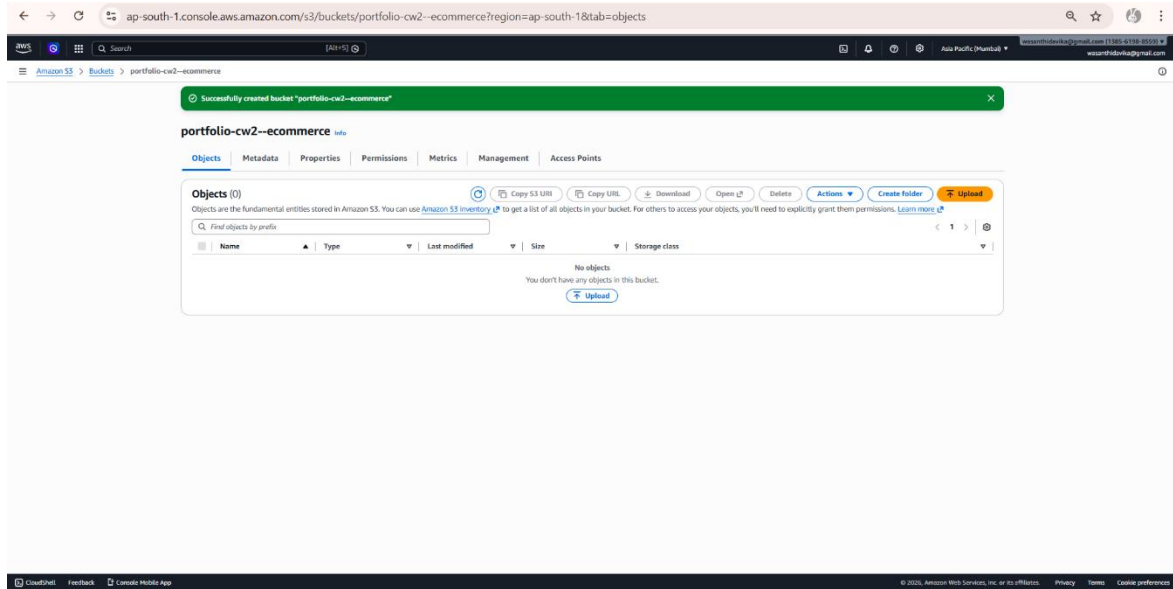


Figure 45 - S3 dashboard showing portfolio-cw2-ecommerce bucket (from docs/ folder)

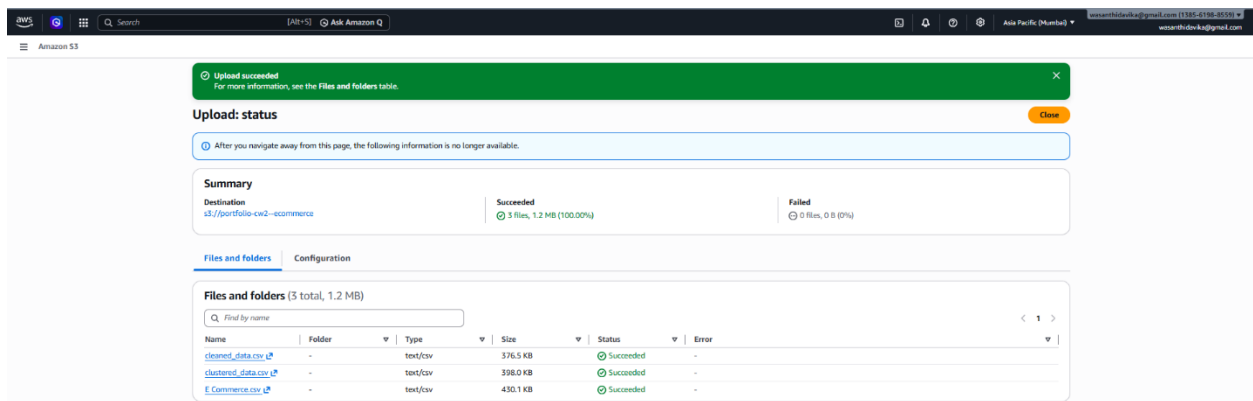


Figure 46 - S3 dashboard showing portfolio-cw2-ecommerce bucket (from docs/ folder)

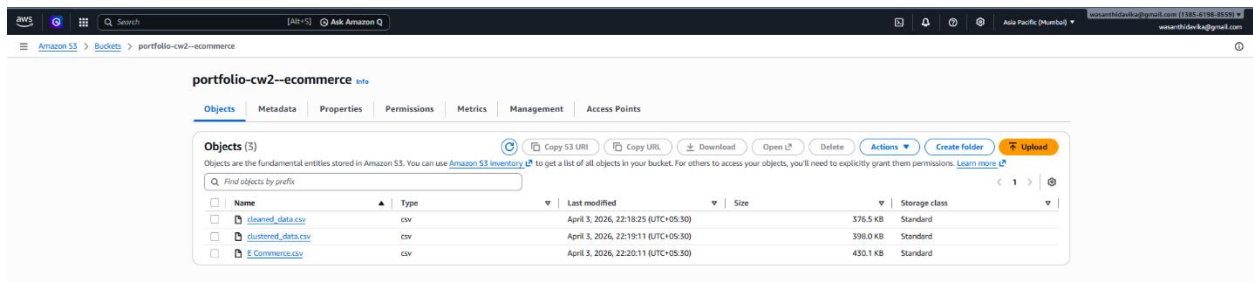


Figure 47 - Screenshot inside bucket showing all 3 CSV files uploaded

7. Collaboration & Documentation

The complete project repository is publicly available at:

<https://github.com/AnuradhaDhananjane/Portfolio-CW2>

7.1 Git Repository Structure

The project is managed using the Git version control, which is hosted on GitHub.. The repository has hierarchical layout that meets the assignment requirements:

- Data: Raw dataset (E Commerce.csv)
- Four Jupyter notebooks: ETL, EDA, Clustering, and Model.
- Outputs: All created charts, cleaned data, and clustered data.
- Docs: Methodology and cloud setup documentation.
- Reports/Final report.
- README.MD — Project Overview and Instructions
- Requirements.txt lists Python dependencies.

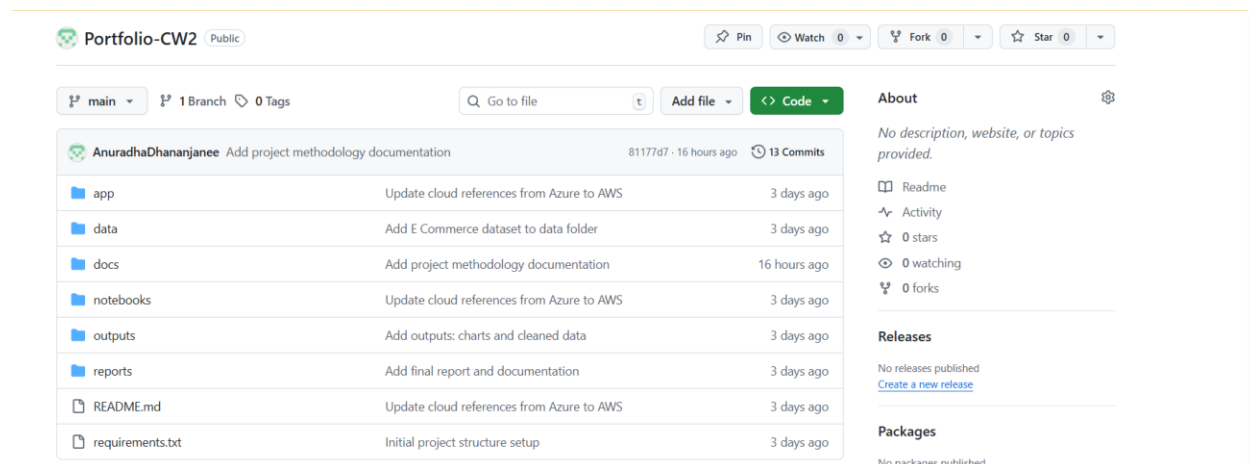


Figure 48 - Github Repo

7.2 Git Commit Strategy

Structured, meaningful commitments were maintained throughout the project.

Table 10 - Git Commit messages

Commit Message	Purpose
Initial project structure setup	Created folder structure and README

Add E Commerce dataset to data folder	Data collection phase
Add notebook 01: ETL and data collection	Data loading and cleaning
Add notebook 02: EDA and visualizations	Exploratory analysis
Add notebook 03: K-Means clustering analysis	Customer segmentation
Add notebook 04: Predictive model and evaluation	Problem solving
Add outputs: charts and cleaned data	Analysis results
Add final report and documentation	Final deliverables
Add AWS cloud setup documentation	Cloud integration

The screenshot displays the GitHub interface for the repository 'AnuradhaDhananjane/Portfolio-CW2'. The 'Commits' tab is selected, showing a list of commits on the 'main' branch. The commits are grouped by date, with the most recent being from April 4, 2026. The commit messages include 'Add project methodology documentation', 'Add AWS cloud setup documentation and screenshots', and a series of updates and additions for notebooks 01 through 04, final report, and dataset setup. Each commit entry shows the commit hash, the author 'AnuradhaDhananjane', and the time since the commit was made.

Commit Message	Commit Hash	Author	Time Ago
Add project methodology documentation	81177d7	AnuradhaDhananjane	15 hours ago
Add AWS cloud setup documentation and screenshots	4ec1834	AnuradhaDhananjane	yesterday
Update cloud references from Azure to AWS	8a8df04	AnuradhaDhananjane	3 days ago
Update cloud references from Azure to AWS	659e418	AnuradhaDhananjane	3 days ago
Add Flask web application frontend and backend	99d418f	AnuradhaDhananjane	3 days ago
Add final report and documentation	2cc6725	AnuradhaDhananjane	3 days ago
Add outputs: charts and cleaned data	61ab786	AnuradhaDhananjane	3 days ago
Add notebook 04: Predictive model and evaluation	a34c5a4	AnuradhaDhananjane	3 days ago
Add notebook 03: K-Means clustering analysis	3a5d228	AnuradhaDhananjane	3 days ago
Add notebook 02: EDA and visualizations	8dcea12	AnuradhaDhananjane	3 days ago
Add notebook 01: ETL and data collection	f6c6ea	AnuradhaDhananjane	3 days ago
Add E Commerce dataset to data folder	1f52fe9	AnuradhaDhananjane	3 days ago
Initial project structure setup	01a90e6	AnuradhaDhananjane	3 days ago

Figure 49 - Git commit

7.3 Assumptions & Decisions

- Assumption: The dataset appropriately reflects real-world shipment data.
- Decision: Random Forest was chosen over Logistic Regression as the primary model because to its superior accuracy.
- Decision: Elbow Method analysis was used with K=3 clusters.
- Decision: Retain outliers for valid business circumstances.
- Assumption: Late_Delivery=1 indicates the package did not arrive on time.

- Deferred cloud integration until submission deadline to optimise free trial usage.

7.4 Challenges & Solutions

Table 11 - Challenges & Solutions

Challenge	Solution
Kernel connection issues in VS Code	Reinstalled ipykernel. Restarted VS Code
Seaborn palette error with dict format	Changed palette - from dict {} to list []
Timing of cloud trials	Completed all coding locally; AWS S3 to be integrated before submission. Azure free tier was not eligible — switched to AWS free tier
Uniqueness of dataset selection	Chose less common dataset to differentiate from other students

8. Conclusion

This study effectively illustrated the entire data analysis lifecycle for a real-world e-commerce business problem. Key achievements include:

- Processed 10,999 genuine shipment records from a public Kaggle dataset.
- Conducted extensive EDA and found that discount amount and shipment weight are the most significant indicators of late delivery.
- Used K-Means clustering to categorise shipments into three relevant business groupings.
- Developed and tested two machine learning models, with Random Forest achieving approximately 69% accuracy.
- Made concrete business ideas to minimise 60% late delivery rate.
- Developed scalable AWS S3 cloud architecture for production deployments.

Despite the inherent difficulty of logistics prediction, the Random Forest model serves as a strong foundation for proactive delivery management. More data collection (weather, traffic, courier performance) can considerably increase model accuracy.

References

Prachi13 (2020). E-Commerce Shipping Dataset. Kaggle.

<https://www.kaggle.com/datasets/prachi13/customer-churn-and-shipping>

Scikit-learn Documentation (2024). RandomForestClassifier. <https://scikit-learn.org/stable/>

Amazon Web Services (2024). AWS S3 Documentation. <https://docs.aws.amazon.com/s3/>

Flask Documentation (2024). <https://flask.palletsprojects.com/>

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, 2825-2830.

2. Question 02

Compare and contrast probability distributions.

1. Introduction

A probability distribution explains the distribution of probabilities among possible values of a random variable. There are broadly two types of statistical distributions, discrete, where the Probability Mass Function (PMF) is applied to provide the probability of countably many outcomes, and continuous, where the Probability Density Function (PDF) is applied to describe probability of an interval of possible values.

This solution gives a comparison and contrast of eight distributions, which include Binomial, Poisson, Geometric, Hypergeometric, Negative Binomial, Exponential, Normal and Chi-Square. The comparison will be organized looking at (i) the individual properties of each distribution, (ii) properties that are common in all distributions, and (iii) major differences and correlations between them.

2. Comparison Table

The characteristics of all the eight distributions are briefly summarized in a table below.

Distribution	Type	Parameters	PMF / PDF	Mean	Variance	Key Scenario	Independent?	n Fixed?	Memoryless?
Binomial	Discrete	n, p	$\binom{n}{x} p^x (1-p)^{n-x}$	np	$np(1-p)$	Successes in n fixed independent trials	Yes	Yes	No
Poisson	Discrete	λ	$\frac{\lambda^x e^{-\lambda}}{x!}$	λ	λ	Events per unit time; rare events, large n	Yes	No (∞)	No
Geometric	Discrete	p	$p(1-p)^x$	q/p	q/p^2	Trials until first success	Yes	No	Yes
Hypergeometric	Discrete	M, K, n	$\frac{\binom{K}{x} \binom{M-K}{n-x}}{\binom{M}{n}}$	nK/M	Complex*	Successes drawn without replacement	No	Yes	No
Neg. Binomial	Discrete	r, p	$\binom{r+x-1}{x} p^r q^x$	rq/p	rq/p^2	Failures until r -th success	Yes	No	No
Exponential	Continuous	λ	$\lambda e^{-\lambda x}, x \geq 0$	$1/\lambda$	$1/\lambda^2$	Waiting time between Poisson events	Yes	No	Yes
Normal	Continuous	μ, σ^2	$\frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$	μ	σ^2	Natural measurements; CLT limit	—	—	No
Chi-square	Continuous	ν (df)	Gamma-based ($x > 0$)	ν	2ν	Sum of squared normals; hypothesis tests	—	—	No

3. Individual Distributions — Definition and Characteristics

a. Binomial Distribution

Type: Discrete

Notation: $X \sim \text{Bin}(n, p)$,

Table 12 - Comparison Table Parameters: n = number of trials, p = probability of success

The Binomial distribution - is a distribution that represents the count of successes in n fixed independent Bernoulli trials where the probability of success is equal in each trial p . It is a direct extension of the Bernoulli distribution - a Bernoulli(p) is $\text{Bin}(1, p)$.

Conditions that must hold:

- Exactly two possible outcomes per trial: success (1) or the failure (0)
- Fixed and known number of trials n
- $P(\text{success}) = p$ is constant for every trial
- All trials independent of each other

PMF: $P(X = x) = \binom{n}{x} \cdot p^x \cdot (1-p)^{(n-x)}$ for $x = 0, 1, 2, \dots, n$

Mean: $E(X) = np$ **Variance:** $\text{Var}(X) = np(1-p) = npq$

Shape: symmetric when $p = 0.5$; rightskewed when $p < 0.5$; left-skewed when $p > 0.5$.

Example: Getting exactly 5 heads in 12 flips of a coin. $X \sim \text{Bin}(12, 0.5)$.

b. Poisson Distribution

Type: Discrete | Notation: $X \sim \text{Po}(\lambda)$ | Parameter: λ = average number of occurrences ($\lambda > 0$)

Poisson distribution describes a number of events that occur in a given interval of time or space in a situation where the events are randomly and independently distributed at a constant rate λ . It is the limiting case of the Binomial when n goes to infinity and p to 0, and the product of the two, $np = \lambda$ remains constant.

Conditions:

- Two possible outcomes per unit: event occurs or does not

- Number of trials n is infinite or unknown
- Probability of success p is very small (approaches 0)
- Trials are independent; only the rate λ is known

PMF: $P(X = x) = (\lambda^x \cdot e^{-\lambda}) / x!$ for $x = 0, 1, 2, \dots$

Mean: $E(X) = \lambda$ **Variance:** $\text{Var}(X) = \lambda$ (unique: mean equals variance)

Examples: *Number of cars passing a junction per hour; insurance claims per week.*

c. Geometric Distribution

Type: Discrete

Notation: $X \sim \text{Geometric}(p)$

Parameter: p = probability of success per trial

When independent Bernoulli trials are repeated, the Geometric distribution shows how many failures there are before the first success. The question here is the opposite of Binomial and Poisson: instead of counting how many successes there are in a set number of trials, we count how long it takes to get the first success.

Conditions:

- Two possible outcomes:
 - Success
 - failure
- Number of trials n is unknown, trials continue until success
- $P(\text{success}) = p$ is constant and independent for every trial

PMF (failures before 1st success): $P(X = x) = p(1-p)^x$ for $x = 0, 1, 2, \dots$

Mean: $E(X) = q/p$ **Variance:** $\text{Var}(X) = q/p^2$ where $q = 1 - p$

Key property — Memorylessness: The Geometric is the only discrete distribution with the memoryless property: $P(X > s+t \mid X > s) = P(X > t)$. Past failures do not affect the probability of future success.

Contrast with Negative Binomial: Geometric is a special case of the Negative Binomial with $r = 1$.

d. Hypergeometric Distribution

Type: Discrete

Notation: $X \sim \text{Hypergeometric}(M, K, n)$

Parameters: M = population size, K = successes in population, n = draws

The Hypergeometric distribution describes how many successes there are in n draws from a finite population of M items that contain K successes, where the items are not replaced. This is the most important difference from the Binomial: since items are not replaced, the chances of success on the next draw change with each draw, which makes trials dependent.

Conditions:

- Two possible outcomes: success or failure
- Fixed and known number of draws n
- $P(\text{success})$ is NOT constant — it changes after each draw
- Trials are NOT independent (no replacement from finite population)

PMF: $P(X = x) = \frac{K C_x \cdot (M-K) C_{(n-x)}}{M C_n}$

Mean: $E(X) = nK/M$ **Variance:** $\text{Var}(X) = n(K/M) ((M-K)/M)((M-n)/(M-1))$

Relationship to Binomial: When M is very large relative to n , Hypergeometric $\approx$ Binomial, because removing items barely affects the remaining probabilities. The factor $(M-n)/(M-1)$ in the variance is called the finite population correction factor.

e. Negative Binomial Distribution

Type: Discrete

Notation: $X \sim \text{NegBin}(r, p)$

Parameters: r = target number of successes, p = probability of success per trial

When independent Bernoulli trials are repeated with a constant probability p , the Negative Binomial distribution models the number of failures before the r -th success. When $r = 1$, the Negative Binomial distribution is exactly the same as the Geometric distribution.

Conditions:

- Two possible outcomes: success or failure
- Number of trials n is unknown — trials continue until the r -th success
- $P(\text{success}) = p$ is constant and independent for every trial

PMF (failures before r -th success): $P(X = x) = \binom{r+x-1}{x} p^r \cdot q^x$

Mean: $E(X) = rq/p$ **Variance:** $\text{Var}(X) = rq/p^2$

Contrast with Binomial: Binomial sets n and counts the number of successes X . Negative Binomial sets the number of successes r and counts the failures X needed to reach that number. The experiment goes backwards.

f. Exponential Distribution

Type: Continuous

Notation: $X \sim \text{Exp}(\lambda)$

Parameter: $\lambda > 0$ (rate parameter)

The Exponential distribution describes how long it takes for events in a Poisson process to happen one after the other.

E.g. The time until the next event happens when events happen at a constant average rate of λ . It is the continuous version of the Geometric distribution.

PDF: $f(x; \lambda) = \lambda e^{-\lambda x}$ for $x \geq 0$ (zero otherwise)

Mean: $E(X) = 1/\lambda$ **Variance:** $\text{Var}(X) = 1/\lambda^2$

Key property — Memorylessness: The only continuous distribution having a memoryless attribute. The Geometric distribution is mirrored in discrete time: no matter how long you've waited, the chance of the next occurrence remains constant.

Relationship to Poisson: Waiting durations between events are exponential (λ), whereas event counts are Poisson (λ). They depict the same procedure in two viewpoints.

Relationship to Gamma: Exponential is a special thing of the Gamma distribution with shape $r = 1$. Sum of r independent $\text{Exponential}(\lambda)$ variables follows $\text{Gamma}(r, \lambda)$.

g. Normal Distribution

Type: Continuous

Notation: $X \sim N(\mu, \sigma^2)$

Parameters: μ = mean (location), σ^2 = variance (shape), $\sigma > 0$

Normal distribution is most essential one in statistics. It creates a fully symmetric, bell-shaped curve centered on μ . It results from the sum of a large number of tiny independent random variables. It is used to represent heights, weights, measurement mistakes, exam results, and any phenomenon formed by the accumulation of numerous small random influences.

PDF: $f(x; \mu, \sigma) = (1/\sqrt{2\pi\sigma^2}) \cdot e^{-(x-\mu)^2/(2\sigma^2)}$ for $-\infty < x < \infty$

Mean: $E(X) = \mu$ **Variance:** $\text{Var}(X) = \sigma^2$

Standard Normal: When $\mu = 0$ and $\sigma = 1$, $Z \sim N(0, 1)$. Any normal variable can be standardised: $Z = (X - \mu)/\sigma$.

Empirical Rule: approximately 68% of the observations fall in $\pm 1\sigma$ of the mean, 95% within $\pm 2\sigma$, and 99.73% within $\pm 3\sigma$.

Approximations: Normal approximates the Binomial (large N) and Poisson (large λ), linking all three families.

h. Chi-Square Distribution

Type: Continuous

Notation: $X \sim \chi^2(v)$

Parameter: v = degrees of freedom (positive integer)

The Chi-Square distribution is simply derived from the Normal distribution. If $Z_1, Z_2, \dots, Z_v$ are independent standard normal variables, then $X = Z_1^2 + Z_2^2 + \dots + Z_v^2$ displays a Chi-Square distribution with v degrees of freedom. It is used to test hypotheses (goodness-of-fit and independence tests) and to calculate confidence ranges for variances.

PDF: $f(x) = [1/(2^{v/2} \cdot \Gamma(v/2))] \cdot x^{(v/2-1)} \cdot e^{(-x/2)}$ for $x > 0$

Mean: $E(X) = v$ **Variance:** $\text{Var}(X) = 2v$ where v = degrees of freedom

Shape: right skewed for small v ; approaches a symmetric bell shape as $v \rightarrow \infty$ by the Central Limit Theorem.

Relationship to Normal: Chi-Square is composed of squared standard normal variables.

Relationship to Gamma: $\chi^2(v)$ is a special case of Gamma with shape = $v/2$ and rate = $1/2$.

Table 13 - Comparison of distributions

Distribution	When to Use	Random Variable X	PMF / PDF	Mean E(X)	Variance Var(X)
Binomial	Fixed number of trials n , independent, constant p	No of successes in n trials	${}^nC_x p^x (1-p)^{n-x}$, $x = 0, 1, 2, \dots, n$	np	$np(1-p)$
Poisson	Events occurring at fixed time/space, $n \rightarrow \infty$, $p \rightarrow 0$, $\lambda = np$	No of events in an interval	$e^{-\lambda} \lambda^x / x!$, $x = 0, 1, 2, \dots$	λ	λ
Geometric	Repeated trials until 1st success	Number of failures before 1st success	$(1-p)^x p$, $x = 0, 1, 2, \dots$	$(1-p)/p$	$(1-p)/p^2$
Hyper geometric	Sampling without replacement - from finite population	Number of successes in n draws	$[{}^KC_x \cdot {}^{M-K}C_{n-x}] / {}^MC_n$, $x = 0, 1, \dots, n$	nK/M	$n(K/M)((M-K)/M)((M-n)/(M-1))$
Negative Binomial	Repeated trials until r-th success	Number of failures.	${}^{x+r-1}C_x p^r (1-p)^x$, $x = 0, 1, 2, \dots$	$r(1-p)/p$	$r(1-p)/p^2$

		before r-th success			
Exponential	Waiting time between Poisson events, continuous, $x \geq 0$	Time until next event	$\lambda e^{-\lambda x}, x \geq 0$	$1/\lambda$	$1/\lambda^2$
Normal	Continuous symmetric data, bell-shaped, $-\infty < x < \infty$	Continuous measurement value	$(1/\sigma\sqrt{2\pi}) e^{-(x-\mu)^2/2\sigma^2}$	μ	σ^2
Chi-Square	Sum of squared standard normal variables, hypothesis testing	Test statistic / sum of squared normals	$[1/(2^{v/2}) \Gamma(v/2)] x^{v/2-1} e^{-x/2}, x > 0$	v	$2v$

4. Similarities Across All Distributions

Before contrasting, it is important to identify what each of the eight distributions share:

- All are probability distributions: They assign the probabilities to the outcomes of a random variable.
- All satisfy the axioms of probability: probabilities are non-negative, and total probability sums/integrates to 1.
- All have a mean $E(X)$ and a variance $Var(X)$ that summaries their central tendency and spread.
- Each has a unique shape, location, or scale that defines it.
- All may be used to calculate probability for real-world random events.

5. Compare and Contrast

5.1 First Major Contrast — Discrete vs Continuous

The most fundamental dividing line is whether the random variable takes countable or interval values.

Discrete distributions (Binomial/ Poisson/ Geometric/Hypergeometric/ Negative Binomial) It uses a Probability Mass Function (PMF). Probability is assigned to each specific value $P(X = x)$, which is nonzero.

Continuous distributions (Exponential, Normal, Chi-Square) It uses a Probability Density Function (PDF). $P(X = x) = 0$ for any exact point; probability is only meaningful over an interval: $P(a < X < b) = \int_a^b f(x) dx$.

5.2 Comparing the Five Discrete Distributions

All five employ Bernoulli trials (which have two outcomes: success or failure). The fundamental variations are in what is fixed and what is tallied.

Table 14 - Comparing the Five Discrete Distributions

Property	Binomial	Poisson	Geometric	Hypergeometric	Neg. Binomial
Trials n	Fixed	Unknown/ ∞	Unknown	Fixed	Unknown
p constant?	Yes	Very small	Yes	No	Yes
Independent?	Yes	Yes	Yes	No	Yes
X counts	Successes in n	Events in interval	Trials to 1st success	Successes w/o replace.	Failures to r-th success
Memoryless?	No	No	YES	No	No

Binomial vs Poisson: These are closely related; Poisson is the limiting case of Binomial when $n \rightarrow \infty$ and $p \rightarrow 0$, with $np = \lambda$ constant. Both count distinct occurrences and demand independence. The essential common trait is that both can simulate the same sort of event-counting scenario. The Poisson has the feature that its mean equals its variance (both = λ), but the Binomial's mean np is always bigger than its variance npq (because $q < 1$).

Binomial vs Hypergeometric: Both have a fixed number of draws (n) and count successes. The key difference is independence. Binomial presupposes sampling with replacement (or from an infinite population), therefore p is constant. Hypergeometric sampling implies no replacement from a limited population M , therefore p varies with each draw. Hypergeometric variance includes a finite population - correction factor $(M-n)/(M-1)$ that Binomial does not have. When M is very big in relation to n , Hypergeometric converges to Binomial. This is an essential commonality demonstrating that the two concepts are equivalent under various sampling assumptions.

Geometric vs Negative Binomial: These are sibling distributions, with both repeating independent trials with a constant p until they attain a success rate. Geometric stops at the first success ($r = 1$), but negative binomial ceases at the r th success. Geometric is only a specific case of Negative Binomial with $r = 1$. Both include an open-ended n . The mean and variance formulas are the same: mean = rq/p and variance = rq/p^2 . Geometric simply substitutes $r = 1$.

Geometric — unique among discrete: The Geometric is only discrete distribution which having memoryless properties. This distinguishes it from Binomial, Poisson, Hypergeometric, and Negative Binomial, which all rely on past results.

Poisson — unique mean = variance: The Poisson distribution is unique in that its mean and variance are identical (both = λ). In every other distribution, the mean and variance are determined by distinct factors and are not always equal.

5.3 Comparing the Three Continuous Distributions

Normal vs Exponential: Both are continuous, but they have quite distinct shapes and domains. The Normal distribution is symmetric and bell-shaped, defined throughout $(-\infty, \infty)$, and centered at μ , while the Exponential is severely right skewed and defined exclusively for $x \geq 0$. Normal models values that can be either above or below a center value, whereas Exponential models positive-valued waiting times. A important common characteristic within this family: the Exponential possesses the memoryless quality, making it the continuous equivalent of the Geometric distribution, but the Normal does not.

Normal vs Chi-Square: The Chi-Square derives directly from the Normal; it is the distribution of a sum of squared standard normal variables. Both are continuous, but the Normal is symmetric while the Chi-Square is right skewed (for small v). As v rises, the Chi-Square gets more symmetric and approaches normalcy according to the Central Limit Theorem. The Normal has two adjustable parameters (μ, σ^2), but Chi-Square has only one (v ,

degrees of freedom). They serve distinct functions: the Normal is used to directly model data, whereas the Chi-Square is largely utilized in statistical inference to test hypotheses regarding categorical data or variances.

Exponential vs Chi-Square: Both are right-skewed and defined solely for positive values. Both are special examples of the Gamma distribution: exponential is Gamma with shape $r = 1$, while Chi-Square is Gamma with shape $v/2$ and rate $1/2$. They are relatives in the Gamma family. However, they serve different purposes: Exponential models wait periods in stochastic processes, whereas Chi-Square models evaluate statistics in hypothesis testing.

5.4 Cross-Family Relationships (Discrete ↔ Continuous)

Poisson ↔ Exponential: The most significant cross-family coupling. If events occur at a Poisson rate (discrete counts per interval), the waiting time between consecutive occurrences is exponential (λ). They depict the same underlying stochastic process from two different perspectives: counting events and timing gaps.

Geometric ↔ Exponential (memoryless analogues): Both have the memoryless property: geometric in discrete time, exponential in continuous time. The Geometric distribution has a continuous-time limit known as the exponential. This is the strongest structural commonality across the discrete-continuous split.

Binomial → Normal (approximation): When n is high and p is not severe ($np > 5$ and $nq > 5$), Normal (np, npq) accurately approximates the Binomial distribution. This is the result of the Central Limit Theorem.

Poisson → Normal (approximation): When λ is large, the Poisson distribution is well approximated by Normal (λ, λ). Again, a consequence of the CLT.

6. Similarities and Differences

Table 15 - Key Similarities and Differences

Dimension	Similarities / Distributions sharing it	Differences / Contrast
Discrete vs Continuous	Binomial, Poisson, Geometric, Hypergeometric, Neg. Binomial	Exponential, Normal, Chi-square
Uses PMF	All five discrete distributions	— (use PDF instead)

Uses PDF	—	All three continuous distributions
Mean = Variance	Poisson only (both equal λ)	—
Symmetric shape	None	Normal only
Memoryless property	Geometric (only discrete)	Exponential (only continuous)
Fixed number of trials n	Binomial, Hypergeometric	Others have unknown/open n
Requires independence	Binomial, Poisson, Geometric, Neg. Binomial	Hypergeometric does NOT
Derived from Normal	—	Chi-square (sum of squared Z's)
Limiting/special cases	Binomial $\rightarrow$ Poisson; Geometric $\rightarrow$ Neg. Binomial ($r=1$)	Exponential $\rightarrow$ Gamma ($r=1$); Chi-square $\rightarrow$ Gamma ($v/2, 1/2$)

Hypergeometric variance includes finite population - correction factor $(M-n)/(M-1)$, which shrinks the variance relative to the Binomial when sampling from a finite population.

7. Conclusion

The eight probability distributions form an interconnected family rather than isolated models. The five discrete distributions (Binomial, Geometric, Poisson, Hypergeometric, and Negative Binomial) are all derived from Bernoulli-type trials, but they differ significantly in whether n is fixed, trials are independent, and the variable X counts. The three continuous distributions (Exponential, Normal, and Chi-Square) represent continuous phenomena, with the Normal acting as the central distribution from which Chi-Square is generated while Binomial and Poisson converge under large-sample circumstances.

The most important structural connections are:

- (1) Poisson is the limit of the Binomial as $n \rightarrow \infty$ and $p \rightarrow 0$;
- (2) Geometric and Exponential are discrete and continuous analogues sharing the memoryless property;
- (3) Hypergeometric converges to Binomial as population size $M \rightarrow \infty$;
- (4) Geometric is a special case of Negative Binomial with $r = 1$;
- (5) Exponential is a special case of the Gamma with $r = 1$; and

(6) Chi-Square is a type of Gamma that is calculated using squared Normal variables.

Understanding these linkages enables the appropriate distribution to be chosen for each given probability circumstance.

References

Gupta, S. C., & Kapoor, V. K. (n.d.). *Fundamentals of mathematical statistics: A modern approach* (10th revised ed.). Sultan Chand & Sons.

Walpole, R. E., Myers, R. H., & Myers, S. L. (2012). *Probability and statistics for engineers and scientists* (9th ed.). Pearson Education.

Hogg, R. V., McKean, J. W., & Craig, A. T. (2019). *Introduction to mathematical statistics* (8th ed.). Pearson Education.

Ross, S. M. (2019). *Introduction to probability and statistics for engineers and scientists* (6th ed.). Academic Press / Elsevier.